Denoising Diffusion-based Generative Modeling: Foundations and Applications

Viet Nguyen

About me

- Previously, I graduated with an Engineer degree from the Global ICT program (K64).
- At university, I was also a member of the Data Science Lab of Assoc. Prof. Khoat Than.
- Now, I am a research resident at VinAI Research.
- My research interests include generative model and its applications (specially Diffusion models).

Reference

Denoising Diffusion-based Generative Modeling CVPR2022 tutorial

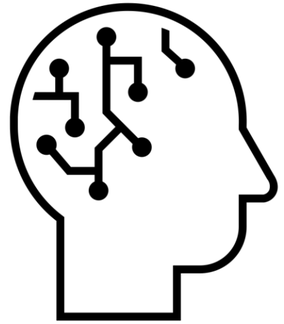
Denoising Diffusion Models: A Generative Learning Big Bang CVPR2023 tutorial

Deep Generative Learning

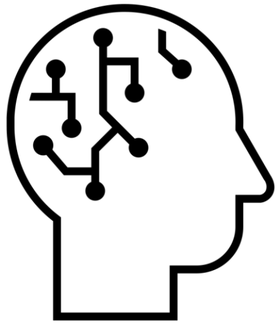
Learning to generate data

Samples from a Data Distribution

Train



Neural Network



Sample

Denoising Diffusion Models

Emerging as powerful generative models, outperforming GANs

"Diffusion Models Beat GANs on Image Synthesis" Dhariwal & Nichol, OpenAI, 2021

"Cascaded Diffusion Models for High Fidelity Image Generation" Ho et al., Google, 2021

Text-to-Image Generation

DALL·E 2

“a teddy bear on a skateboard in times square”

[“Hierarchical Text-Conditional Image Generation with CLIP Latents” Ramesh et al., 2022]

Imagen

A group of teddy bears in suit in a corporate office celebrating the birthday of their friend. There is a pizza cake on the desk.

[“Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding”, Saharia et al., 2022]

Outline

- Part (1): Denoising Diffusion Probabilistic Models
- Part (2): *On Inference Stability for Diffusion Models (our work)*
- Part (3): Conditional Generation and Guidance
- Part (4): Text-to-image Diffusion Models

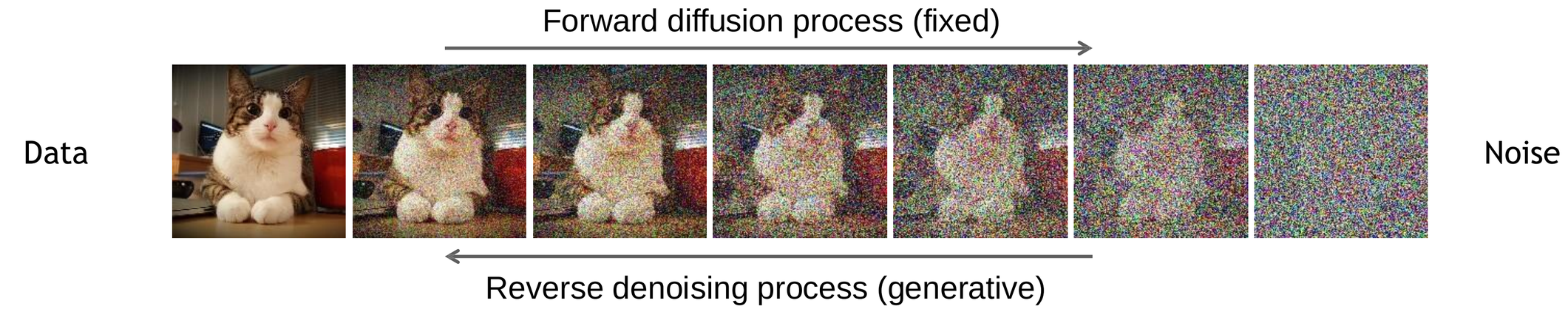
Part (1):
Denoising Diffusion Probabilistic Models

Denoising Diffusion Models

Learning to generate by denoising

Denoising diffusion models consist of two processes:

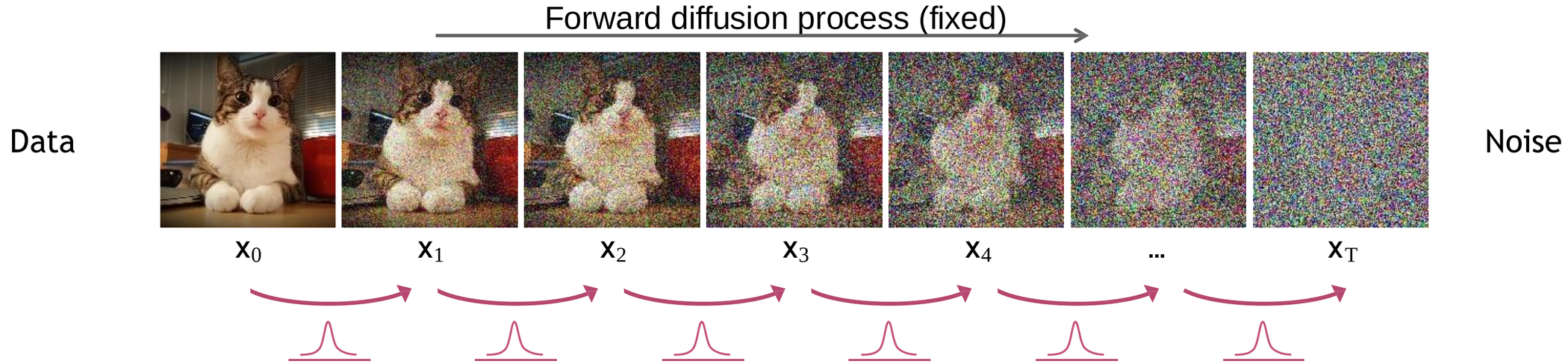
- Forward diffusion process that gradually adds noise to input
- Reverse denoising process that learns to generate data by denoising



Sohl-Dickstein et al., Deep Unsupervised Learning using Nonequilibrium Thermodynamics, ICML 2015
Ho et al., Denoising Diffusion Probabilistic Models, NeurIPS 2020
Song et al., Score-Based Generative Modeling through Stochastic Differential Equations, ICLR 2021

Forward Diffusion Process

The formal definition of the forward process in T steps:



The "forward diffusion" process:
add Gaussian noise each step

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x_t}; \sqrt{1-\beta_t}\mathbf{x_{t-1}}, \beta_t\mathbf{I})$$

➡ $$q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^{T} q(\mathbf{x}_t|\mathbf{x}_{t-1})$$ (Probability Chain Rule - Markov Chain)

Diffusion Kernel

https://lilianweng.github.io/posts/2021-07-11-diffusion-models/

Data

$\mathbf{x}_0$

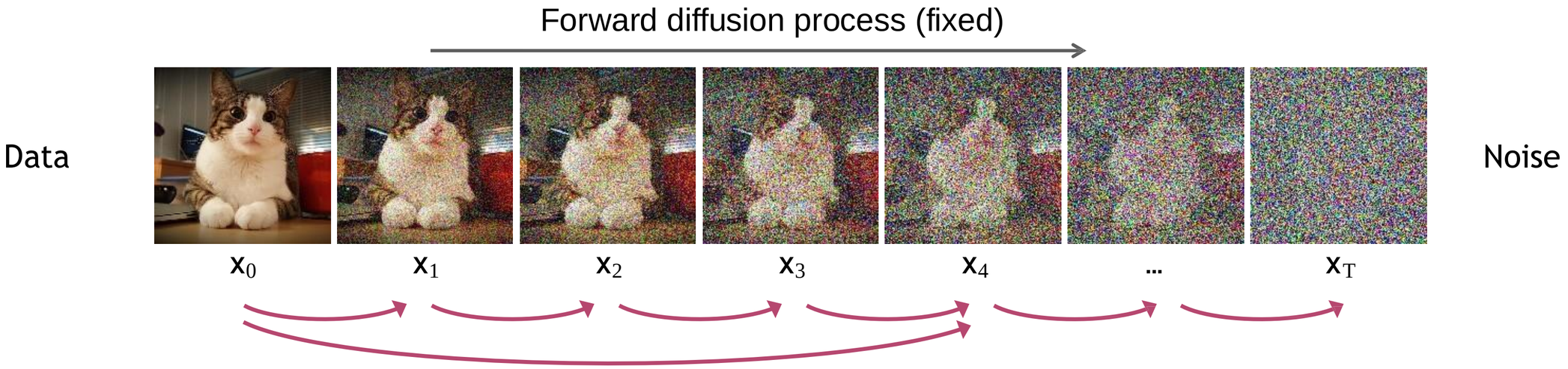
$$
\begin{aligned}
\mathbf{x}_t &= \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{1-\alpha_t}\boldsymbol{\epsilon}_{t-1} && \text{;where } \boldsymbol{\epsilon}_{t-1}, \boldsymbol{\epsilon}_{t-2}, \cdots \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \\
&= \sqrt{\alpha_t\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1-\alpha_t\alpha_{t-1}}\bar{\boldsymbol{\epsilon}}_{t-2} && \text{;where } \bar{\boldsymbol{\epsilon}}_{t-2} \text{ merges two Gaussians (*).} \\
&= \cdots \\
&= \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon} \\
q(\mathbf{x}_t|\mathbf{x}_0) &= \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I})
\end{aligned}
$$

(*) Recall that when we merge two Gaussians with different variance, $\mathcal{N}(\mathbf{0}, \sigma_1^2\mathbf{I})$ and $\mathcal{N}(\mathbf{0}, \sigma_2^2\mathbf{I})$, the new distribution is $\mathcal{N}(\mathbf{0}, (\sigma_1^2 + \sigma_2^2)\mathbf{I})$. Here the merged standard deviation is $\sqrt{(1-\alpha_t) + \alpha_t(1-\alpha_{t-1})} = \sqrt{1-\alpha_t\alpha_{t-1}}$.

Nice property: samples from an *arbitrary forward step* are also Gaussian-distributed!

Define $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$ ➡ $q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I}))$ (Diffusion Kernel)

Diffusion Kernel



Nice property: samples from an *arbitrary forward step* are also Gaussian-distributed!

Define $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$ ➡ $q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I}))$ (Diffusion Kernel)

Gaussian reparameterization trick:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{(1-\bar{\alpha}_t)}\,\epsilon \quad \text{where} \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

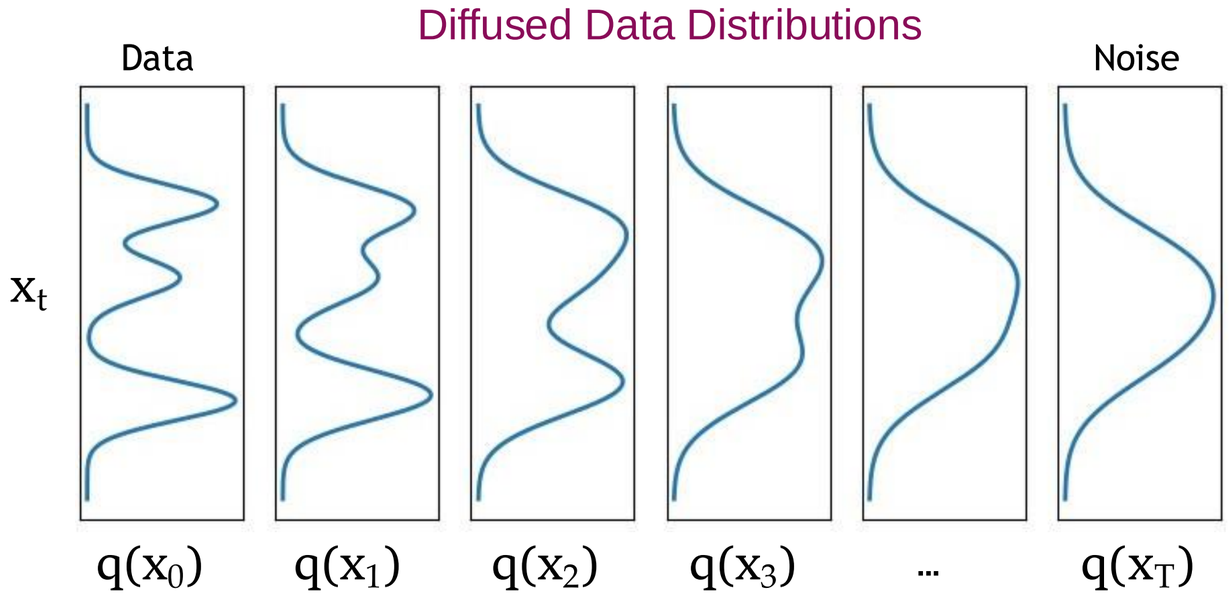
β_t values schedule (i.e., the noise schedule) is designed such that $\bar{\alpha}_T \to 0$ and $q(\mathbf{x}_T|\mathbf{x}_0) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}))$

What happens to a distribution in the forward diffusion?

So far, we discussed the diffusion kernel $q(\mathbf{x}_t|\mathbf{x}_0)$ but what about $q(\mathbf{x}_t)$?

$$\underbrace{q(\mathbf{x}_t)}_{\text{Diffused data dist.}} = \int \underbrace{q(\mathbf{x}_0, \mathbf{x}_t)}_{\text{Joint dist.}} d\mathbf{x}_0 = \int \underbrace{q(\mathbf{x}_0)}_{\text{Input data dist.}} \underbrace{q(\mathbf{x}_t|\mathbf{x}_0)}_{\text{Diffusion kernel}} d\mathbf{x}_0$$

The diffusion kernel is Gaussian convolution.



We can sample $\mathbf{x}_t \sim q(\mathbf{x}_t)$ by first sampling $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ and then sampling $\mathbf{x}_t \sim q(\mathbf{x}_t|\mathbf{x}_0)$ (i.e., ancestral sampling).

Generative Learning by Denoising

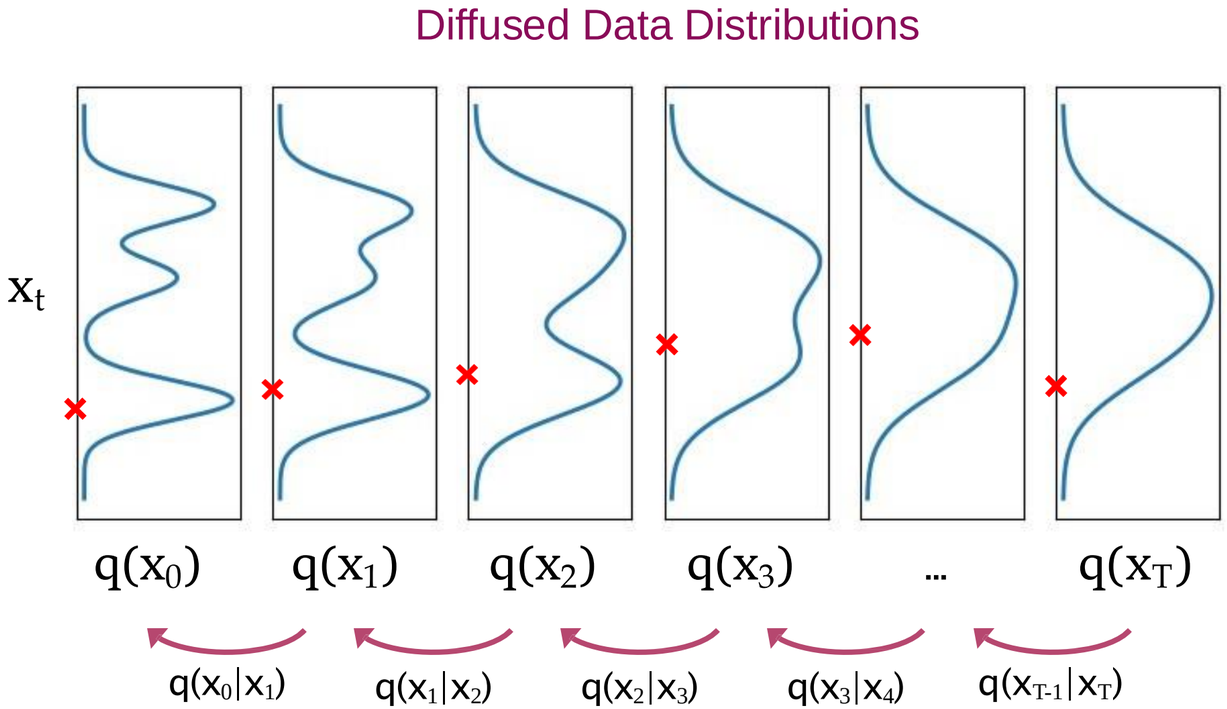
Recall, that the diffusion parameters are designed such that $q(\mathbf{x}_T) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}))$

Generation:

Sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$

Iteratively sample $\mathbf{x}_{t-1} \sim \underbrace{q(\mathbf{x}_{t-1}|\mathbf{x}_t)}$

True Denoising Dist.

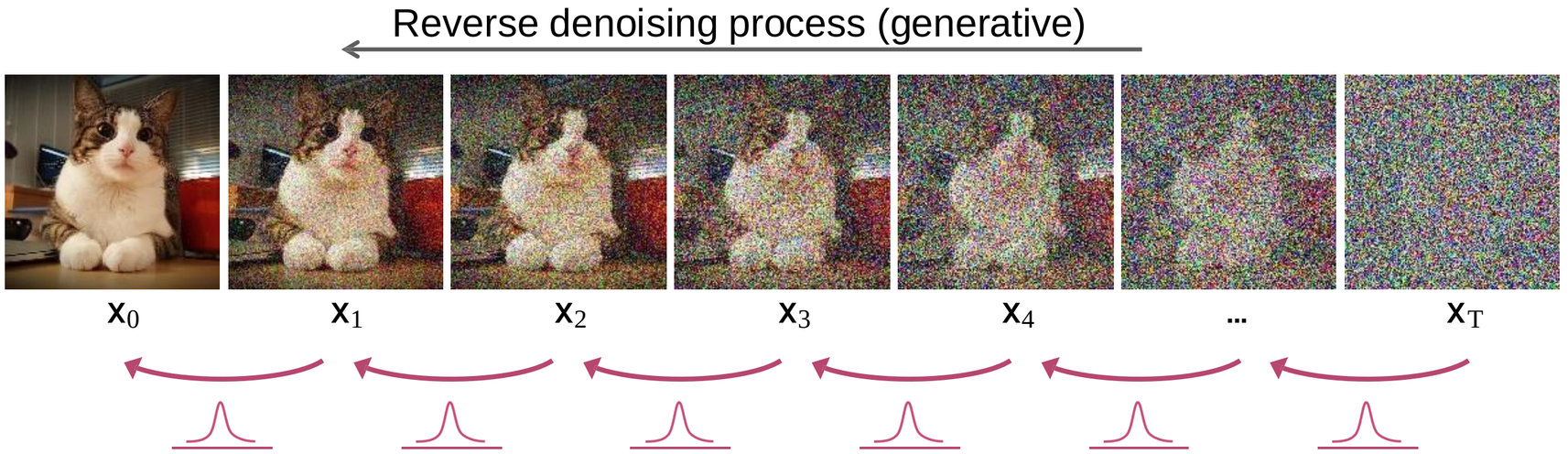


In general, $q(\mathbf{x}_{t-1}|\mathbf{x}_t) \propto q(\mathbf{x}_{t-1})q(\mathbf{x}_t|\mathbf{x}_{t-1})$ is intractable.

Can we approximate $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$? Yes, we can use a Normal distribution if β_t is small in each forward diffusion step.

Reverse Denoising Process

Formal definition of forward and reverse processes in T steps:



$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$$

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \underbrace{\mu_\theta(\mathbf{x}_t, t)}, \sigma_t^2\mathbf{I})$$

Trainable network
(U-net, Denoising Autoencoder)

$$\Rightarrow \quad p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

Learning Denoising Model

High-level intuition

Because $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is intractable, we can instead derive a tractable posterior distribution $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$

Then we can train a neural network $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ to match this distribution.

The learning objection is to minimize:

$$D_{\mathrm{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))$$

Learning Denoising Model

Tractable posterior distribution

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I})$$

Using Bayes' rule:

$$\begin{aligned}
q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) &= q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)\frac{q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)} \\
&\propto \exp\Big(-\frac{1}{2}\big(\frac{(\mathbf{x}_t - \sqrt{\alpha_t}\mathbf{x}_{t-1})^2}{\beta_t} + \frac{(\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0)^2}{1-\bar{\alpha}_{t-1}} - \frac{(\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0)^2}{1-\bar{\alpha}_t}\big)\Big) \\
&= \exp\Big(-\frac{1}{2}\big(\frac{\mathbf{x}_t^2 - 2\sqrt{\alpha_t}\mathbf{x}_t\mathbf{x}_{t-1} + \alpha_t\mathbf{x}_{t-1}^2}{\beta_t} + \frac{\mathbf{x}_{t-1}^2 - 2\sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0\mathbf{x}_{t-1} + \bar{\alpha}_{t-1}\mathbf{x}_0^2}{1-\bar{\alpha}_{t-1}} - \frac{(\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0)^2}{1-\bar{\alpha}_t}\big)\Big) \\
&= \exp\Big(-\frac{1}{2}\big((\frac{\alpha_t}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}})\mathbf{x}_{t-1}^2 - (\frac{2\sqrt{\alpha_t}}{\beta_t}\mathbf{x}_t + \frac{2\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}}\mathbf{x}_0)\mathbf{x}_{t-1} + C(\mathbf{x}_t, \mathbf{x}_0)\big)\Big)
\end{aligned}$$

Following the standard Gaussian density function, we can derive the mean and the variance:

$$\begin{aligned}
\tilde{\beta}_t &= 1/(\frac{\alpha_t}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}}) = 1/(\frac{\alpha_t - \bar{\alpha}_t + \beta_t}{\beta_t(1-\bar{\alpha}_{t-1})}) = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t} \cdot \beta_t \\
\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) &= (\frac{\sqrt{\alpha_t}}{\beta_t}\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}}\mathbf{x}_0)/(\frac{\alpha_t}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}}) \\
&= (\frac{\sqrt{\alpha_t}}{\beta_t}\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}}\mathbf{x}_0)\frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t} \cdot \beta_t \\
&= \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\mathbf{x}_0 \\
&= \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\frac{1}{\sqrt{\bar{\alpha}_t}}(\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}_t) = \frac{1}{\sqrt{1-\beta_t}}\Big(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_t\Big)
\end{aligned}$$

Learning Denoising Model

Variational upper bound

For training, we can form variational upper bound that is commonly used for training VAEs:

$$
\begin{aligned}
-\log p_\theta(\mathbf{x}_0) &\leq -\log p_\theta(\mathbf{x}_0) + D_{\mathrm{KL}}(q(\mathbf{x}_{1:T}|\mathbf{x}_0)\|p_\theta(\mathbf{x}_{1:T}|\mathbf{x}_0)) \\
&= -\log p_\theta(\mathbf{x}_0) + \mathbb{E}_{\mathbf{x}_{1:T}\sim q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})/p_\theta(\mathbf{x}_0)}\right] \\
&= -\log p_\theta(\mathbf{x}_0) + \mathbb{E}_q\left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} + \log p_\theta(\mathbf{x}_0)\right] \\
&= \mathbb{E}_q\left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})}\right] \\
\text{Let } L &= \mathbb{E}_{q(\mathbf{x}_{0:T})}\left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})}\right] \geq -\mathbb{E}_{q(\mathbf{x}_0)} \log p_\theta(\mathbf{x}_0)
\end{aligned}
$$

Learning Denoising Model

Variational upper bound

Sohl-Dickstein et al. ICML 2015 and Ho et al. NeurIPS 2020 show that:

$$
\begin{aligned}
L &= \mathbb{E}_{q(\mathbf{x}_{0:T})}\Big[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})}\Big] \\
&= \mathbb{E}_q\Big[\log \frac{\prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_T)\prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}\Big] \\
&= \mathbb{E}_q\Big[-\log p_\theta(\mathbf{x}_T) + \sum_{t=1}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}\Big] \\
&= \mathbb{E}_q\Big[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)}\Big] \\
&= \mathbb{E}_q\Big[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \Big(\frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}\cdot\frac{q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0)}\Big) + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)}\Big] \\
&= \mathbb{E}_q\Big[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \sum_{t=2}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)}\Big] \\
&= \mathbb{E}_q\Big[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \log \frac{q(\mathbf{x}_T|\mathbf{x}_0)}{q(\mathbf{x}_1|\mathbf{x}_0)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)}\Big] \\
&= \mathbb{E}_q\Big[\log \frac{q(\mathbf{x}_T|\mathbf{x}_0)}{p_\theta(\mathbf{x}_T)} + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} - \log p_\theta(\mathbf{x}_0|\mathbf{x}_1)\Big] \\
&= \mathbb{E}_q[\underbrace{D_{\mathrm{KL}}(q(\mathbf{x}_T|\mathbf{x}_0)\parallel p_\theta(\mathbf{x}_T))}_{L_T} + \sum_{t=2}^T \underbrace{D_{\mathrm{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)\parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} \underbrace{-\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0}]
\end{aligned}
$$

Total loss

$$
\begin{aligned}
L &= L_T + L_{T-1} + \cdots + L_0 \\
\text{where } L_T &= D_{\mathrm{KL}}(q(\mathbf{x}_T|\mathbf{x}_0)\parallel p_\theta(\mathbf{x}_T)) \\
L_{t-1} &= D_{\mathrm{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)\parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)) \text{ for } 2\le t\le T \\
L_0 &= -\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)
\end{aligned}
$$

Parameterizing the Denoising Model

Since both $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ and $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ are Normal distributions, the KL divergence has a simple form:

$$L_{t-1} = D_{\mathrm{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)) = \mathbb{E}_q \left[\frac{1}{2\sigma_t^2} ||\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)||^2 \right] + C$$

Recall that $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\, \mathbf{x}_0 + \sqrt{(1-\bar{\alpha}_t)}\, \epsilon$. Ho et al. NeurIPS 2020 observe that:

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{1-\beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon \right)$$

They propose to represent the mean of the denoising model using a *noise-prediction* network:

$$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{1-\beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right)$$

With this parameterization

$$L_{t-1} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\frac{\beta_t^2}{2\sigma_t^2 (1-\beta_t)(1-\bar{\alpha}_t)} ||\epsilon - \epsilon_\theta(\underbrace{\sqrt{\bar{\alpha}_t}\, \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon}_{\mathbf{x}_t}, t)||^2 \right] + C$$

Training Objective Weighting

Trading likelihood for perceptual quality

$$L_{t-1} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\underbrace{\frac{\beta_t^2}{2\sigma_t^2(1-\beta_t)(1-\bar{\alpha}_t)}}_{\lambda_t} ||\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}\ \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\ \epsilon, t)||^2 \right]$$

The time dependent λ_t ensures that the training objective is weighted properly for the maximum data likelihood training.

However, this weight is often very large for small t's.

Ho et al. NeurIPS 2020 observe that simply setting $\lambda_t = 1$ improves sample quality. So, they propose to use:

$$L_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), t \sim \mathcal{U}(1,T)} \left[||\epsilon - \epsilon_\theta(\underbrace{\sqrt{\bar{\alpha}_t}\ \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\ \epsilon}_{\mathbf{x}_t}, t)||^2 \right]$$

For more advanced weighting see Choi et al., Perception Prioritized Training of Diffusion Models, CVPR 2022.

Summary

Training and Sample Generation

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

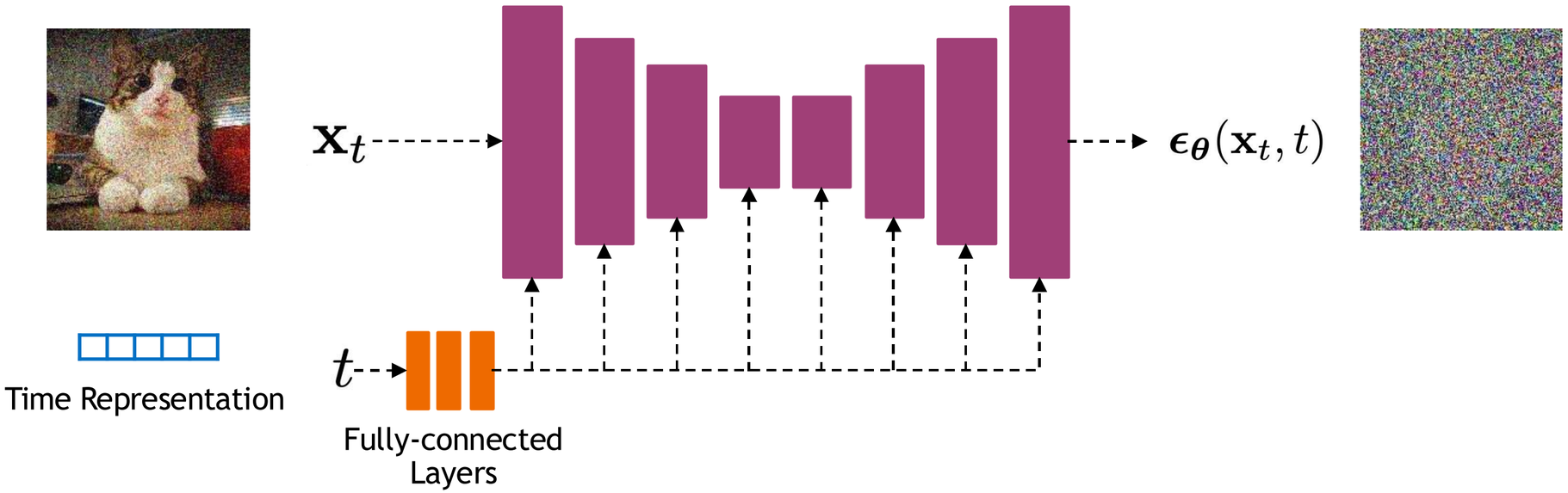
Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Implementation Considerations

Network Architectures

Diffusion models often use U-Net architectures with ResNet blocks and self-attention layers to represent $\epsilon_\theta(\mathbf{x}_t, t)$



Time representation: sinusoidal positional embeddings or random Fourier features.

Time features are fed to the residual blocks using either simple spatial addition or using adaptive group normalization layers. (see Dhariwal and Nichol NeurIPS 2021)

Diffusion Parameters

Noise Schedule

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x_t}; \sqrt{1-\beta_t}\mathbf{x_{t-1}}, \beta_t\mathbf{I})$$

Data

Noise

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \sigma_t^2\mathbf{I})$$

Above, β_t and σ_t^2 control the variance of the forward diffusion and reverse denoising processes respectively.

Often a linear schedule is used for β_t, and σ_t^2 is set equal to β_t.

Kingma et al. NeurIPS 2022 introduce a new parameterization of diffusion models using signal-to-noise ratio (SNR), and show how to learn the noise schedule by minimizing the variance of the training objective.

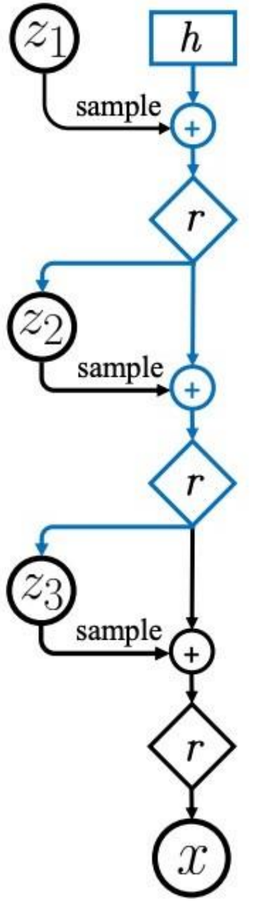
We can also train σ_t^2 while training the diffusion model by minimizing the variational bound (Improved DPM by Nichol and Dhariwal ICML 2021) or after training the diffusion model (Analytic-DPM by Bao et al. ICLR 2022).

Connection to VAEs

Diffusion models can be considered as a special form of hierarchical VAEs.

However, in diffusion models:

- The encoder is fixed
- The latent variables have the same dimension as the data
- The denoising model is shared across different timestep
- The model is trained with some reweighting of the variational bound.



Vahdat and Kautz, NVAE: A Deep Hierarchical Variational Autoencoder, NeurIPS 2020
Sønderby, et al.. Ladder variational autoencoders, NeurIPS 2016.

Summary

Denoising Diffusion Probabilistic Models

In this part, we reviewed denoising diffusion probabilistic models.

The model is trained by sampling from the forward diffusion process and training a denoising model to predict the noise. We discussed how the forward process perturbs the data distribution or data samples.

The devil is in the details:

- Network architectures
- Objective weighting
- Diffusion parameters (i.e., noise schedule)

See "Elucidating the Design Space of Diffusion-Based Generative Models" by Karras et al. for important design decisions.

Practice

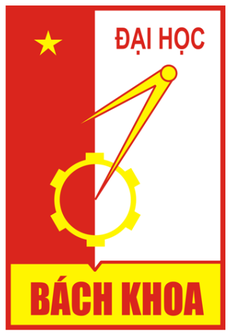
Diffusion Models Tutorial Google Colab

Part (2):
On Inference Stability for Diffusion Models

Oral presentation at AAAI 2024 (Top 2%)

Viet Nguyen*, Giang Vu*, Tung Nguyen Thanh, Khoat Than↓, Toan Tran

(*: equal contribution, ↓: corresponding author)



Diffusion Models and conventional training

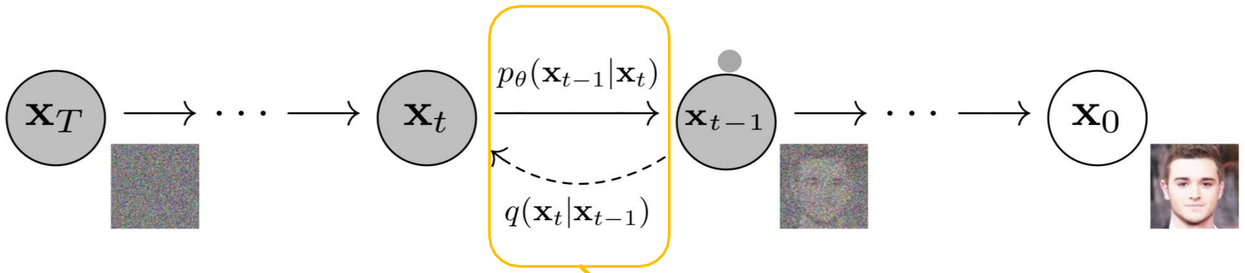


Figure from "Denoising Diffusion Probabilistic Models", Ho et al., NeurIPS 2020

$$\mathbb{E}_q\Big[\underbrace{D_{\mathrm{KL}}(q(\mathbf{x}_T|\mathbf{x}_0)\parallel p(\mathbf{x}_T))}_{L_T}+\sum_{t>1}\underbrace{D_{\mathrm{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)\parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}}\underbrace{-\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0}\Big]$$

Algorithm 1: Conventional training

Require: Empirical data distribution q, number T of timesteps, the noise predictor $\boldsymbol{f}_\theta$, learning rate η.

repeat

$\boldsymbol{x}_0 \sim q(\boldsymbol{x}_0)$

$t \sim \mathrm{Uniform}(\{1,\dots,T\})$

$\epsilon \sim \mathcal{N}(\mathbf{0},\mathbf{I})$

$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$

$\mathcal{L}_{simple} = \|\boldsymbol{f}_\theta(\boldsymbol{x}_t,t) - \boldsymbol{\epsilon}_t\|^2$

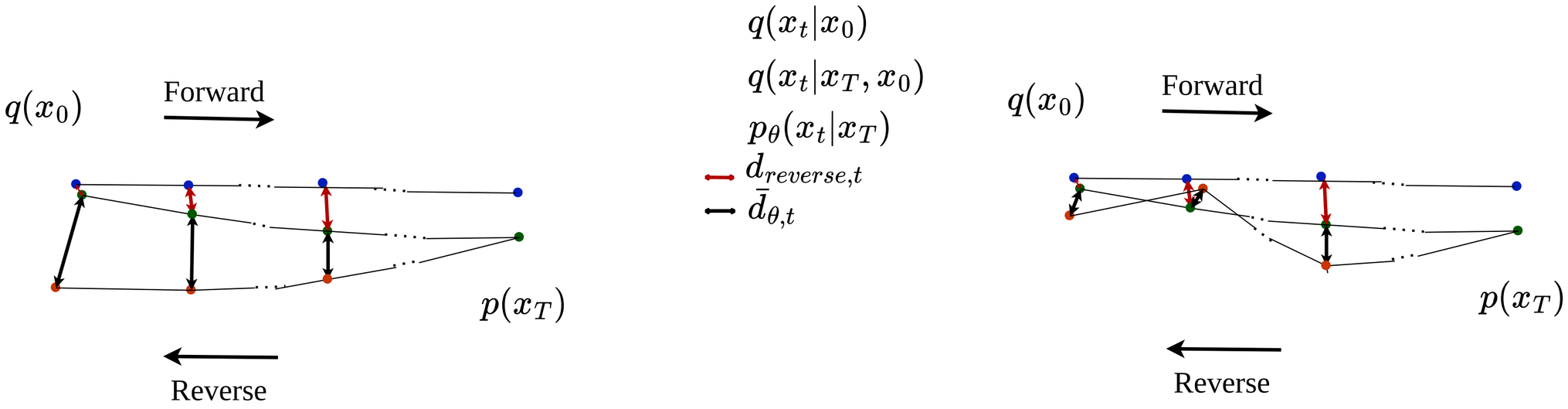
$\theta \leftarrow \theta - \eta \bigtriangledown_\theta \mathcal{L}_{simple}$

until converged

- Consider each timestep in isolation.
- Optimize each transition to the optimum.

Estimation gap

How is the inference trajectory actually? How does a single error affect the whole process?



Here, $\bar{d}_{\theta,t}$ denotes the total error at timestep t in the sampling process. This value can diminish at the end of process if errors at consecutive timesteps cancel each other out.

Estimation gap

How to measure the total error of inference process?

Single gap:
(incurred by the noise predictor)

$$d_{\theta,t} = \gamma_{1,t} \frac{\sqrt{1-\bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}} (\boldsymbol{f}_\theta(\boldsymbol{x}_t, t) - \boldsymbol{\epsilon}_t).$$

The term "single gap" denotes the estimation error that arises when predicting the posterior distribution $q(x_{t-1}|x_t, x_0)$ by $p_\theta(x_{t-1}|x_t)$.

Cumulative gap:
(accounts all previous gaps in the sampling trajectory)

$$\bar{d}_{\theta,t} = d_{\theta,t} + \sum_{i=t+1}^{T} \left[\prod_{s=t}^{i-1} \gamma_{2,s} \right] d_{\theta,i}$$

Estimation gap:
(cumulative gap at timestep 2)

$$d_\theta(\boldsymbol{x}_0) = \sum_{i=2}^{T} \tau_i (\boldsymbol{f}_\theta(\boldsymbol{x}_i, i) - \boldsymbol{\epsilon}_i),$$

Ultimate Goal: Minimize the estimation gap

** Check out the paper for details*

Method: Sequence-aware loss function

How to minimize that term more efficiently?

Estimation gap:
(cumulative gap at timestep 2)

$$d_\theta(\boldsymbol{x}_0) = \sum_{i=2}^{T} \tau_i(\boldsymbol{f}_\theta(\boldsymbol{x}_i, i) - \boldsymbol{\epsilon}_i),$$

Findings:

- **It is not feasible to calculate this quantity at each training step.**
- **Grouping some consecutive steps brings about the same effect.**

K-local gap:
(connect K consecutive steps)

$$d^K_{\theta,t} = \sum_{s=t}^{t+K-1} \tau_s(\boldsymbol{f}_\theta(\boldsymbol{x}_s, s) - \boldsymbol{\epsilon}_s).$$

Method: Sequence-aware loss function

Sequence-aware loss function: $\mathcal{L}_{sa} = \mathbf{E}_{t,\boldsymbol{x}_0,\boldsymbol{\epsilon}_{t:t+K-1}} \left\| \frac{1}{K} \sum_{s=t}^{t+K-1} \tau_s(\boldsymbol{f}_\theta(\boldsymbol{x}_s, s) - \boldsymbol{\epsilon}_s) \right\|^2$

Algorithm 3: Sequence-aware training

Require: Data distribution q, number of timesteps T, the noise predictor $\boldsymbol{f}_\theta$, number of consecutive steps K, hyper-parameter λ, learning rate η.

repeat

  $\boldsymbol{x}_0 \sim q(\boldsymbol{x}_0)$

  $t \sim \text{Uniform}(\{1, \dots, T\})$

  for $k \in \{0, \dots, K-1\}$ **do**

    $\boldsymbol{\epsilon}_{t+k} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

    $\boldsymbol{x}_{t+k} = \sqrt{\bar{\alpha}_{t+k}}\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_{t+k}}\boldsymbol{\epsilon}_{t+k}$

  end for

  $\mathcal{L}_{simple} = \|\boldsymbol{f}_\theta(\boldsymbol{x}_t, t) - \boldsymbol{\epsilon}_t\|^2$

  $\mathcal{L}_{sa} = \frac{1}{K^2} \| \sum_{s=t}^{t+K-1} \tau_s(\boldsymbol{f}_\theta(\boldsymbol{x}_s, s) - \boldsymbol{\epsilon}_s) \|^2$

  $\mathcal{L} = \mathcal{L}_{simple} + \lambda \mathcal{L}_{sa}$

  $\theta \leftarrow \theta - \eta \bigtriangledown_\theta \mathcal{L}$

until converged

Method: Sequence-aware loss function

How good is proposed loss function in comparison with the vanilla objective?

Theorem 2 *Let $\boldsymbol{f}_\theta(\boldsymbol{x}_s, s)$ be any noise predictor with parameter θ. Consider the weighted conventional loss function $\mathcal{L}^{\tau}_{simple} := \mathbf{E}_{t,\boldsymbol{x}_0,\boldsymbol{\epsilon}_t}\left[\tau_t^2 \|\boldsymbol{f}_\theta(\boldsymbol{x}_t, t) - \boldsymbol{\epsilon}_t\|^2\right]$, where τ_t is defined in Theorem 1 and $t \in \{2, ..., T\}$. Then*

$$\frac{T-1}{T+K}\mathcal{L}^{\tau}_{simple} \geq \mathcal{L}_{sa} \geq \frac{1}{(T+K)^2}\mathcal{L}_\theta. \qquad (7)$$

The sequence-aware loss function, with any K value, is a tighter upper bound of the estimation gap in comparison with the weighted vanilla objective. The efficiency of SA loss in minimizing estimation gap is later presented in the experiment section.

Experiment results

Dataset	# timesteps T	Method							
		B	SA	B+A	SA+A	B+NPR	SA+NPR	B+SN	SA+SN
CIFAR10 32×32 DDPM (LS)	10	41.41	**30.51**	34.19	**21.66**	32.35	**21.10**	24.06	**19.53**
	50	15.98	**9.24**	7.20	**4.20**	6.18	**3.90**	4.63	**3.61**
	100	11.79	**6.73**	5.31	**3.43**	4.52	**3.25**	3.67	**3.10**
	200	9.15	**5.47**	3.92	**3.28**	3.57	**3.16**	3.31	**3.06**
	1000	5.92	**4.33**	3.98	**3.72**	4.10	**3.84**	3.65	**3.56**
CIFAR10 32×32 DDPM (CS)	10	34.98	**24.59**	23.41	**16.66**	19.94	**14.77**	**16.33**	17.23
	50	11.05	**6.27**	5.42	**3.78**	5.31	**3.67**	4.17	**3.97**
	100	8.25	**4.98**	4.45	**3.53**	4.52	**3.51**	3.83	**3.64**
	200	6.69	**4.40**	4.04	**3.53**	4.10	**3.54**	3.72	**3.61**
	1000	4.95	**4.05**	4.26	**3.84**	4.27	**3.87**	4.07	**3.83**
CelebA 64×64 DDPM	10	36.69	**32.15**	28.99	**27.08**	28.37	**26.73**	**20.60**	26.22
	50	18.96	**17.59**	11.23	**9.43**	10.89	**9.42**	7.88	**7.01**
	100	14.31	**12.77**	8.08	**6.53**	8.23	**6.84**	5.89	**5.18**
	200	10.48	**9.14**	6.51	**5.02**	7.03	**5.49**	5.02	**4.04**
	1000	5.95	**4.69**	5.21	**3.99**	5.33	**4.00**	4.42	**3.56**
CelebA 64×64 DDIM	10	20.54	**12.88**	15.62	**10.52**	14.98	**10.48**	**10.20**	19.29
	50	9.33	**7.01**	6.13	**4.18**	6.04	**4.25**	3.83	**3.19**
	100	6.60	**4.81**	4.29	**3.02**	4.27	**3.13**	3.04	**2.62**
	200	4.96	**3.69**	3.46	**2.61**	3.59	**2.76**	2.85	**2.49**
	1000	3.40	**2.98**	3.13	**2.74**	3.15	**2.78**	2.90	**2.66**

FID score (↓). Here B and SA denote the baseline and our proposed method. A, NPR, and SN denote Analytic-DPM [1], NPR-DPM [2], and SN-DPM [2], respectively.

[1] Bao, F.; Li, C.; Zhu, J.; and Zhang, B. 2022b. AnalyticDPM: an Analytic Estimate of the Optimal Reverse Variance in Diffusion Probabilistic Models. In International Conference on Learning Representations.

[2] Bao, F.; Li, C.; Sun, J.; Zhu, J.; and Zhang, B. 2022a. Estimating the Optimal Covariance with Imperfect Mean in Diffusion Probabilistic Models. In International Conference on Machine Learning, 1555–1584. PMLR.

Experiment results

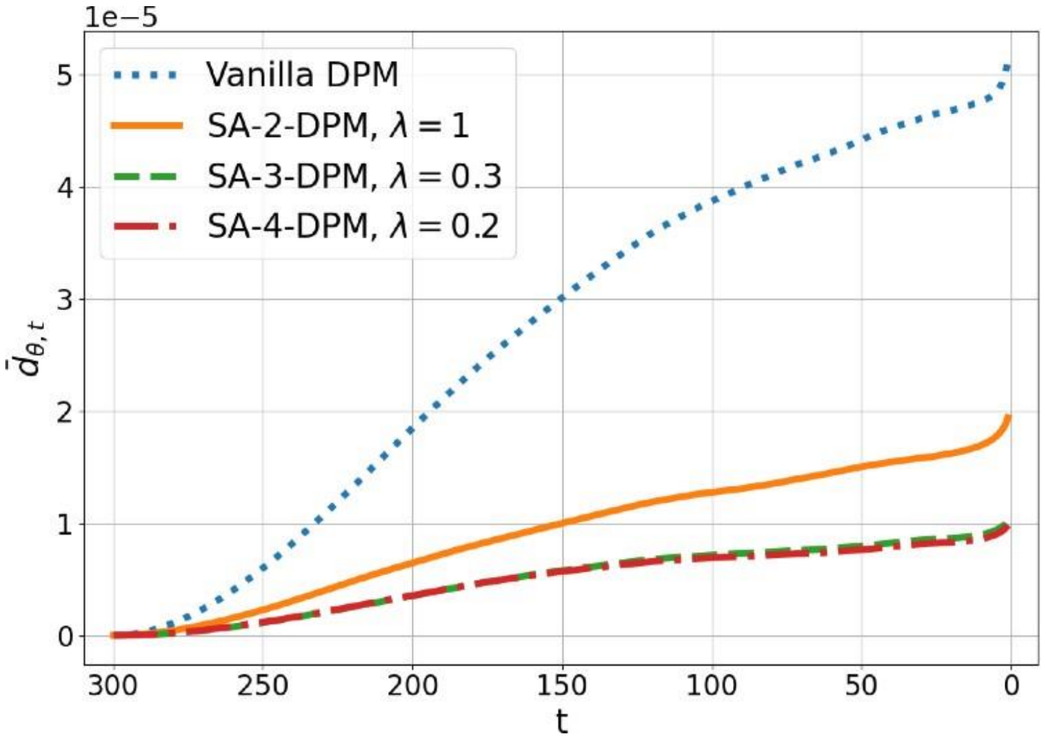
Dataset	# timesteps T	Method							
		B	SA	B+A	SA+A	B+NPR	SA+NPR	B+SN	SA+SN
CIFAR10 32×32 DDPM (LS)	10	6.93	**7.55**	8.05	**8.50**	8.17	**8.53**	8.10	**8.42**
	50	8.34	**8.82**	9.53	**9.62**	9.51	**9.63**	9.49	**9.65**
	100	8.59	**9.04**	9.59	**9.74**	9.55	**9.70**	9.47	**9.73**
	200	8.81	**9.15**	9.59	**9.72**	9.49	**9.62**	9.50	**9.65**
	1000	9.03	**9.24**	9.17	**9.37**	9.18	**9.35**	9.24	**9.41**
CIFAR10 32×32 DDPM (CS)	10	7.48	**7.97**	8.05	**8.37**	8.21	**8.49**	8.47	**8.48**
	50	8.53	**9.09**	8.97	**9.43**	9.02	**9.45**	9.10	**9.46**
	100	8.71	**9.20**	9.07	**9.52**	9.09	**9.53**	9.16	**9.54**
	200	8.84	**9.31**	9.14	**9.55**	9.15	**9.54**	9.18	**9.54**
	1000	8.94	**9.45**	9.04	**9.52**	9.04	**9.52**	9.06	**9.54**

IS metric (↑). Here B and SA denote the baseline and our proposed method. A, NPR, and SN denote Analytic-DPM, NPR-DPM, and SN-DPM, respectively.

Experiment results

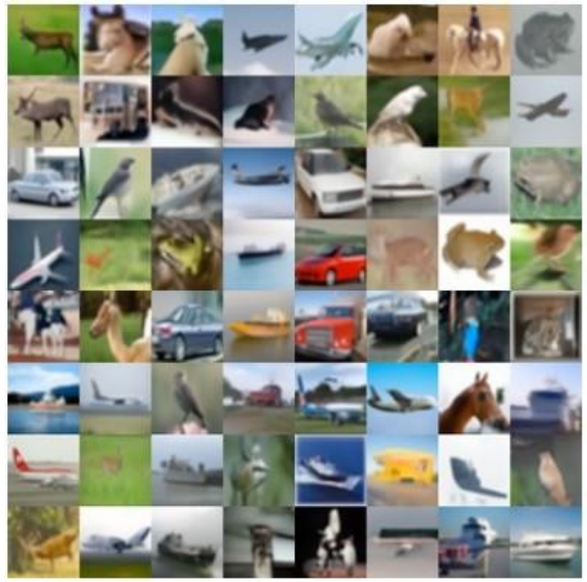
Method	λ	# timesteps T				
		10	50	100	200	1000
DDPM	0	41.41	15.98	11.79	9.15	5.92
SA-2-DPM	0.5	35.39	12.09	8.52	6.56	5.25
	1.0	30.51	9.24	6.73	5.47	4.33
	2.0	19.14	10.59	11.21	12.34	14.20
SA-3-DPM	0.3	30.49	10.27	7.63	6.44	5.47
	0.6	23.71	9.07	7.96	7.77	8.06
	1.5	15.59	11.76	13.90	16.34	19.49
SA-4-DPM	0.2	32.93	10.78	7.78	6.17	4.73
	0.4	26.68	9.33	7.53	7.00	6.95

Ablation study on λ

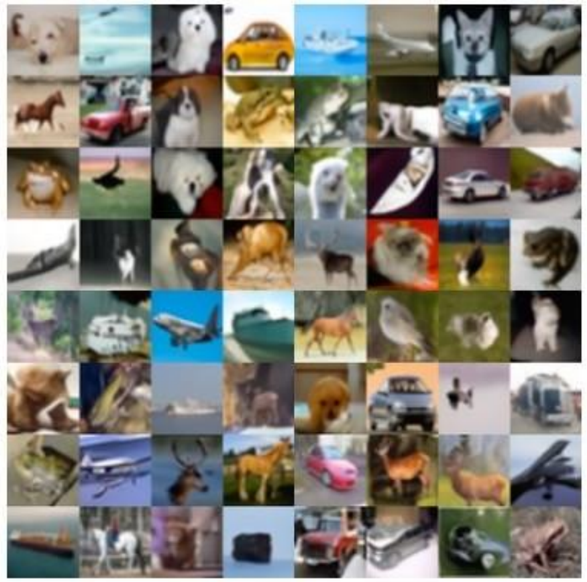


Total gap term when sampling image starting from x_{300} on CIFAR10 dataset.

Experiment results



(a) Vanilla-DDPM



(b) SA-2-DDPM

(c) SA-3-DDPM

(d) SA-4-DDPM

Generated images on CIFAR10 (T=50)

Experiment results

(a) SA-2-DDPM ($T = 10$)

(b) SA-2-DDIM ($T = 10$)

(c) SA-2-DDPM ($T = 50$)

(d) SA-2-DDIM ($T = 50$)

Generated images on CelebA-HQ 256x256

Conclusions

Present a novel sequence-aware loss.

Provide better image quality in a small number of sampling timesteps.

Require longer training time.

Part (3):
Conditional Generation and Guidance

Impressive Results

DALL·E 2

“a propaganda poster depicting a cat dressed as french emperor napoleon holding a piece of cheese”

IMAGEN

“A photo of a raccoon wearing an astronaut helmet, looking out of the window at night.”

“Hierarchical Text-Conditional Image Generation with CLIP Latents” Ramesh et al., 2022

“Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding”, Saharia et al., 2022

How to Control Diffusion Models

Explicit Conditioning

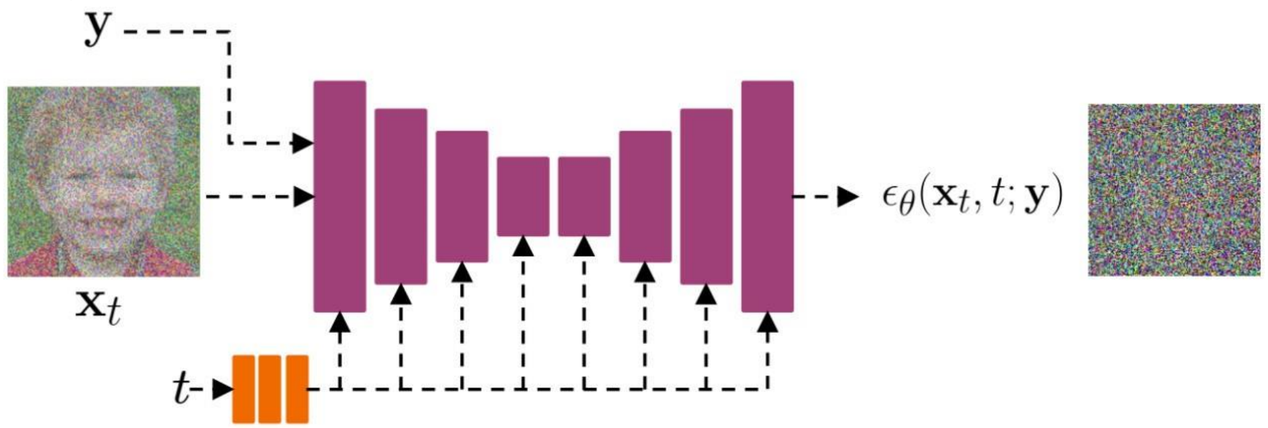
Classifier Guidance

Classifier-Free Guidance

How to Control Diffusion Models

Explicit Conditioning

"a young schoolboy in a red shirt"



Explicit Conditioning

How do we train this?

Use an Image-Text dataset (for example, LAION 5B)

Loss function:

$$\mathbb{E}_{(\mathbf{x},\mathbf{y})\sim p_{\text{data}}(\mathbf{x},\mathbf{y})}\mathbb{E}_{\epsilon\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\mathbb{E}_{t\sim\mathcal{U}[0,T]}\ ||\epsilon_\theta(\mathbf{x}_t, t; \mathbf{y}) - \epsilon||_2^2$$

How to Control Diffusion Models

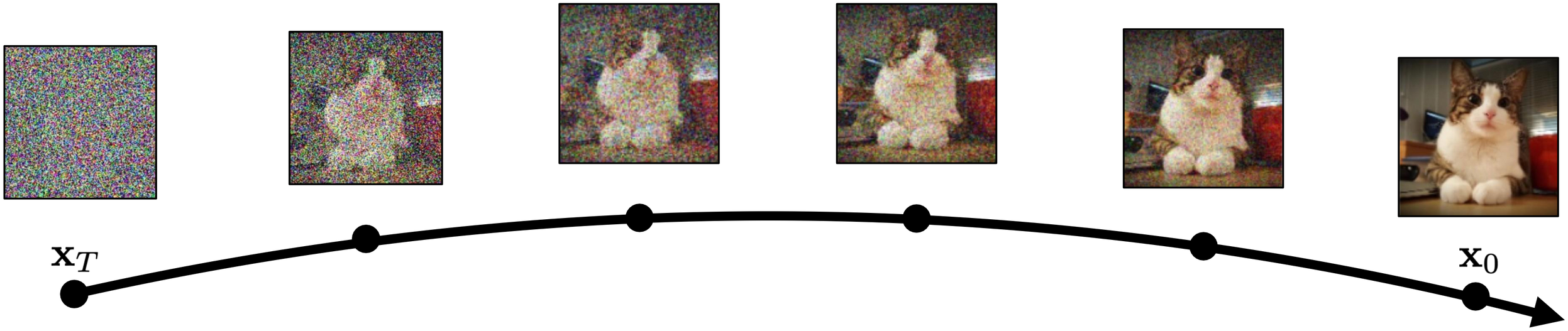
Explicit Conditioning

Classifier Guidance

Classifier-Free Guidance

Classifier Guidance

Diffusion goes from noise to real images step-by-step



Idea: Perturb the Denoising Trajectory

Classifier Guidance: Bayes' Rule in Action

Recall that when sampling from a conditional model $p(\mathbf{x}|\mathbf{y})$ we need an estimation of $\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t|\mathbf{y})$

Using Bayes' rule, we have:

Classifier gradient

Score model

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t|\mathbf{y}) = \nabla_{\mathbf{x}_t} \log \frac{p(\mathbf{y}|\mathbf{x}_t)p(\mathbf{x}_t)}{p(\mathbf{y})} = \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t)$$

Introduce a guidance scale (w):

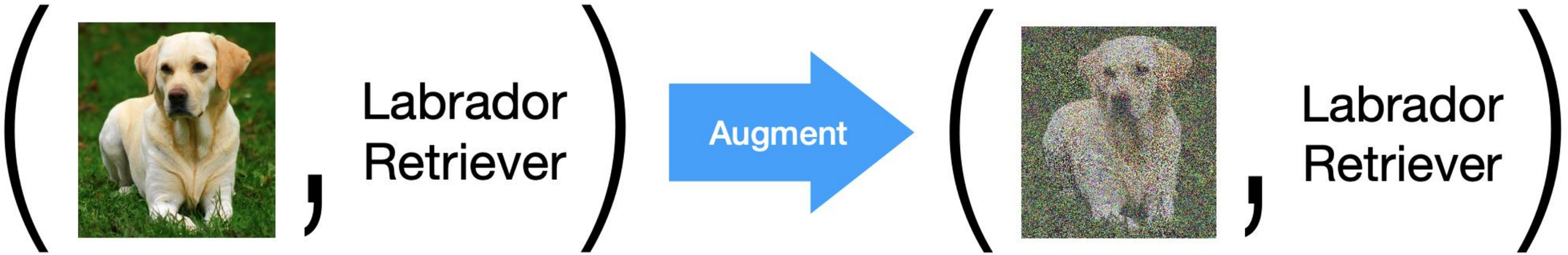
$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t|\mathbf{y}) \approx w\nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t)$$

$$\left(\; p(\mathbf{x}_t|\mathbf{y}) \propto p(\mathbf{y}|\mathbf{x}_t)^w p(\mathbf{x}_t) \; \right)$$

Classifier Guidance

Problem: Classifier isn't trained on noisy images!

Solution: Finetune the classifier on noisy images



Classifier Guidance

Guidance Weight 1.0

Guidance Weight 10.0

Dhariwal and Nichol, "Diffusion Models Beat GANs on Image Synthesis"

Problems with Classifier Guidance

- Need to fine-tune or re-train a classiier on **noisy** data
- Need a pre-trained classification model
  - Classifier gradients are poor. They can suffer from "shortcuts"
- What if we want to use *any* text prompt as input?

How to Control Diffusion Models

Classifier Free Guidance

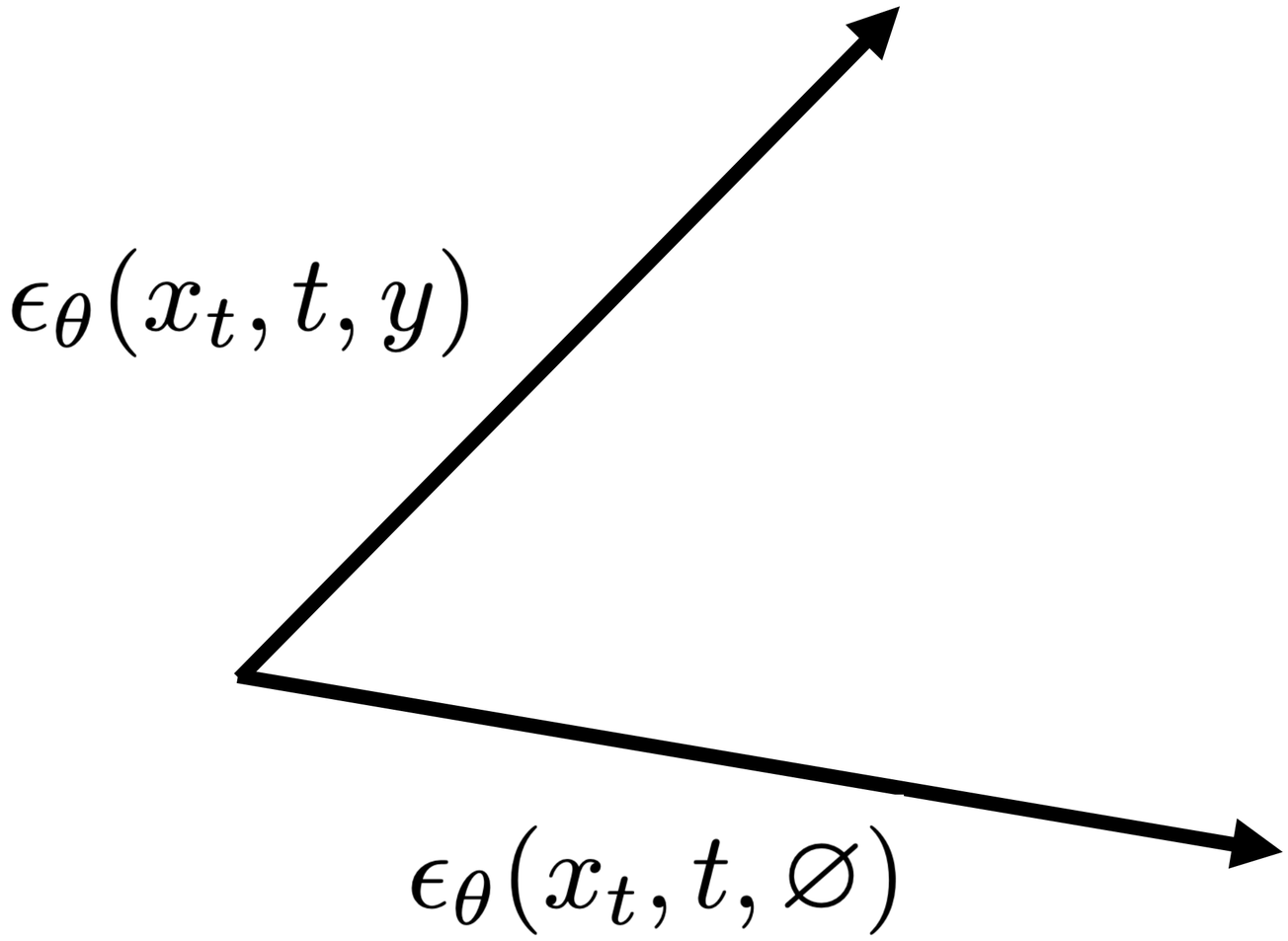
Idea: Use the diffusion model itself to get perturbations for guidance

Train an explicitly conditioned diffusion model: $\epsilon_\theta(x_t, t, y)$

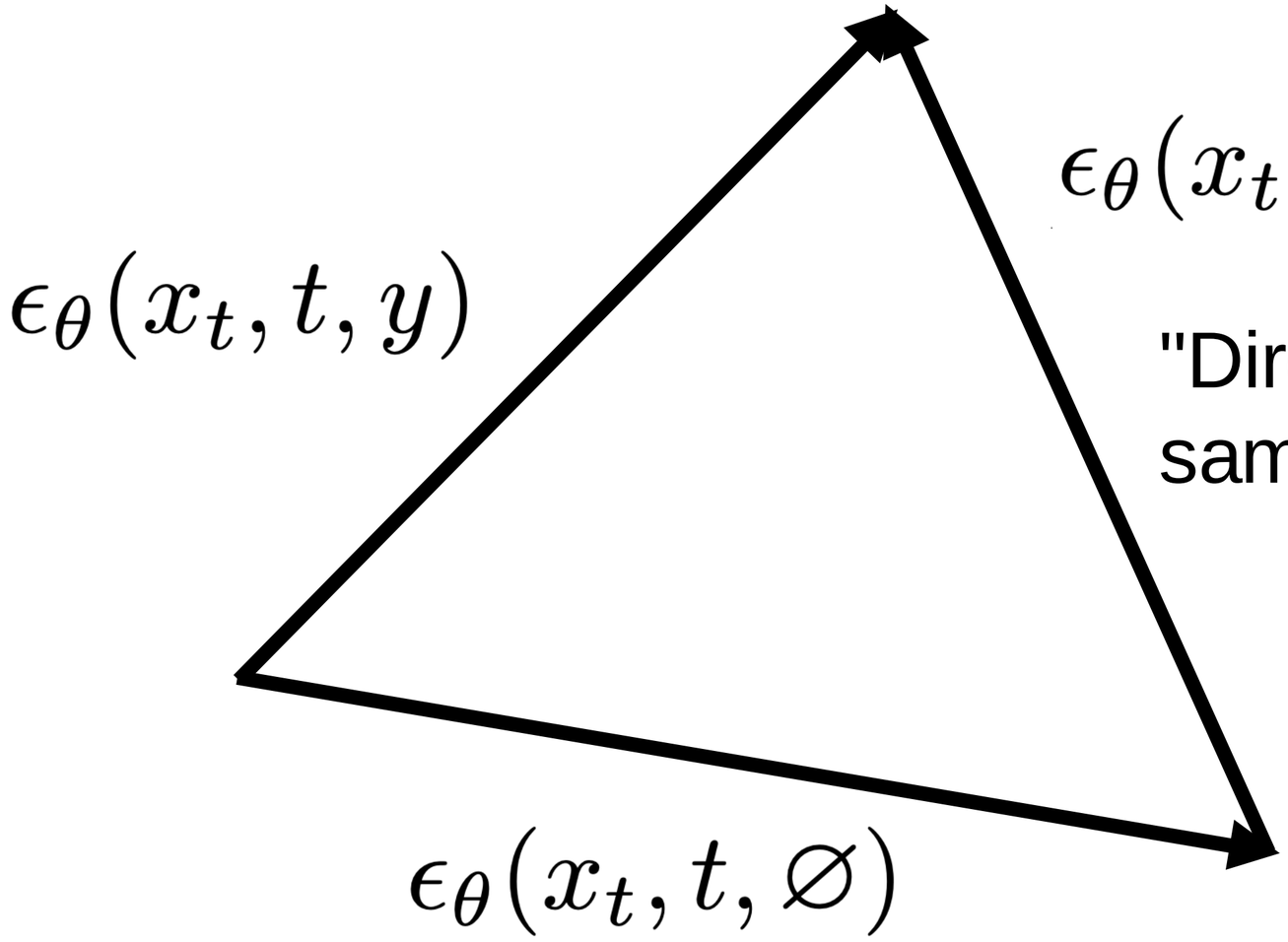
But also train it to be **unconditional**

We can do this with *conditioning dropout:* $\epsilon_\theta(x_t, t, \varnothing)$

Classifier Free Guidance



Classifier Free Guidance



$$\epsilon_\theta(x_t, t, y) - \epsilon_\theta(x_t, t, \varnothing)$$

"Direction" from unconditional sample to conditional sample

Use this as our guidance perturbation

Classifier Free Guidance

Our new noise estimate will then be:

$$\tilde{\epsilon}(x_t, t, y) = \epsilon_\theta(x_t, t, \varnothing) + \gamma(\epsilon_\theta(x_t, t, y) - \epsilon_\theta(x_t, t, \varnothing))$$

"Direction" from unconditional to conditional

Classifier Free Guidance

"A stained glass window of a panda eating bamboo"

$\gamma = 1$

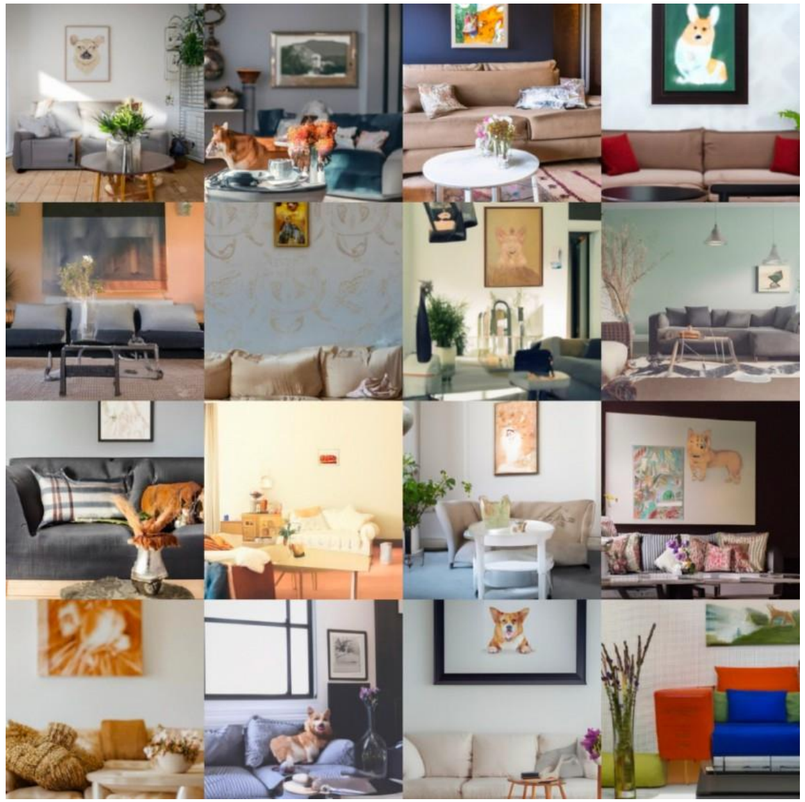
Equivalent to explicit conditioning.
No guidance

$\gamma = 3$

Nichol et al. "GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models"

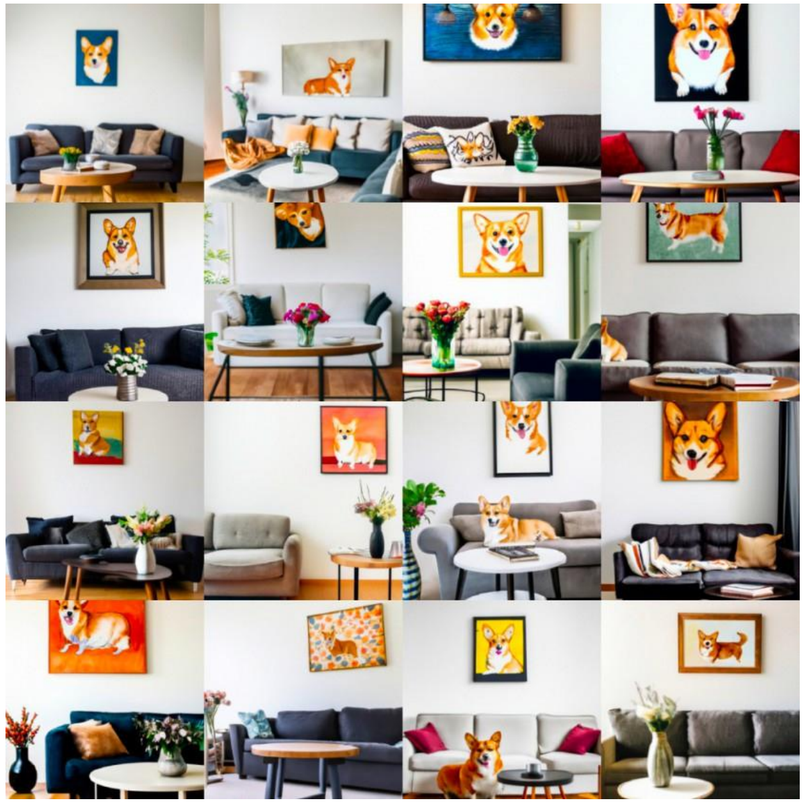
Classifier Free Guidance

"A cozy living room with a painting of a corgi on the wall above a couch and a round coffee table in front of a couch and a vase of flowers on a coffee table "



$\gamma = 1$

Equivalent to explicit conditioning.
No guidance



$\gamma = 3$

Nichol et al. "GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models"

More Resources

- Lilian Weng Tutorial: https://lilianweng.github.io/posts/2021-07-11-diffusion-models/
- Guidance Tutorial by Sander Dieleman: https://sander.ai/2022/05/26/guidance.html

Part (4):
Text-to-image Diffusion Models

GLIDE

OpenAI

- A 64x64 base model + a 64x64 → 256x256 super-resolution model.
- Tried classifier-free and CLIP guidance. Classifier-free guidance works better than CLIP guidance.

"a hedgehog using a calculator"

"a corgi wearing a red bowtie and a purple party hat"

"robots meditating in a vipassana retreat"

"a fall landscape with a small cottage next to a lake"

Samples generated with classifier-free guidance (256x256)

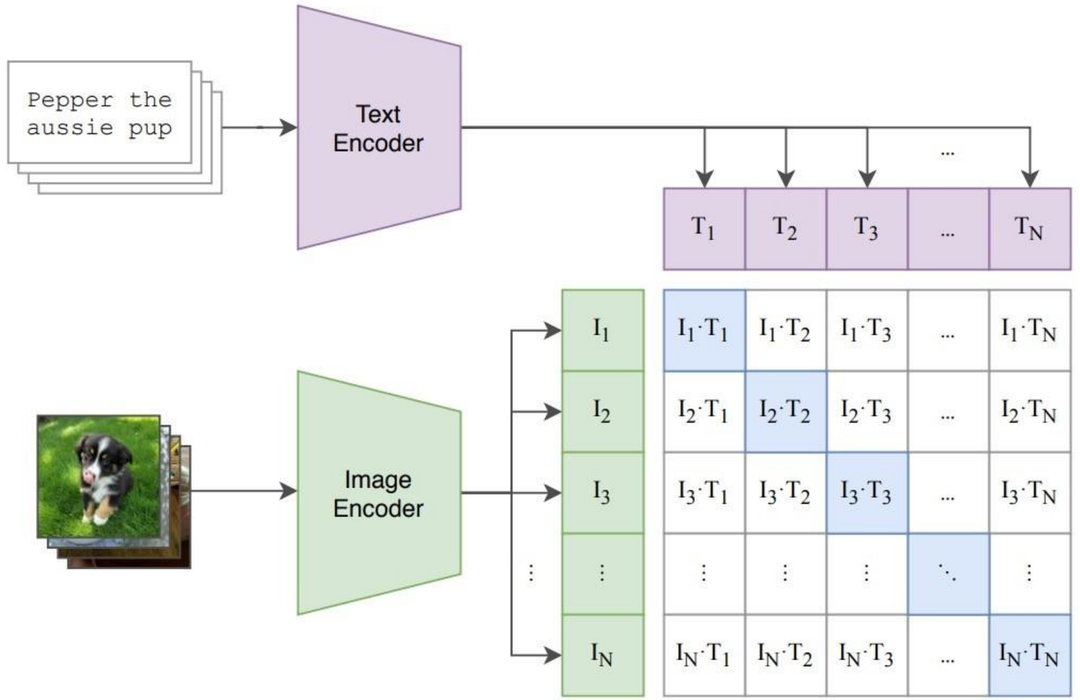
Nichol et al., "GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models", 2021.

CLIP guidance

What is a CLIP model?

- Trained by contrastive cross-entropy loss:

$$-\log \frac{\exp(f(\mathbf{x}_i) \cdot g(\mathbf{c}_j)/\tau)}{\sum_k \exp(f(\mathbf{x}_i) \cdot g(\mathbf{c}_k)/\tau)} - \log \frac{\exp(f(\mathbf{x}_i) \cdot g(\mathbf{c}_j)/\tau)}{\sum_k \exp(f(\mathbf{x}_k) \cdot g(\mathbf{c}_j)/\tau)}$$



- The optimal value of $f(\mathbf{x}) \cdot g(\mathbf{c})$ is

$$\log \frac{p(\mathbf{x}, \mathbf{c})}{p(\mathbf{x})p(\mathbf{c})} = \log p(\mathbf{c}|\mathbf{x}) - \log p(\mathbf{c})$$

Radford et al., “Learning Transferable Visual Models From Natural Language Supervision”, 2021.
Nichol et al., “GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models”, 2021.

CLIP guidance

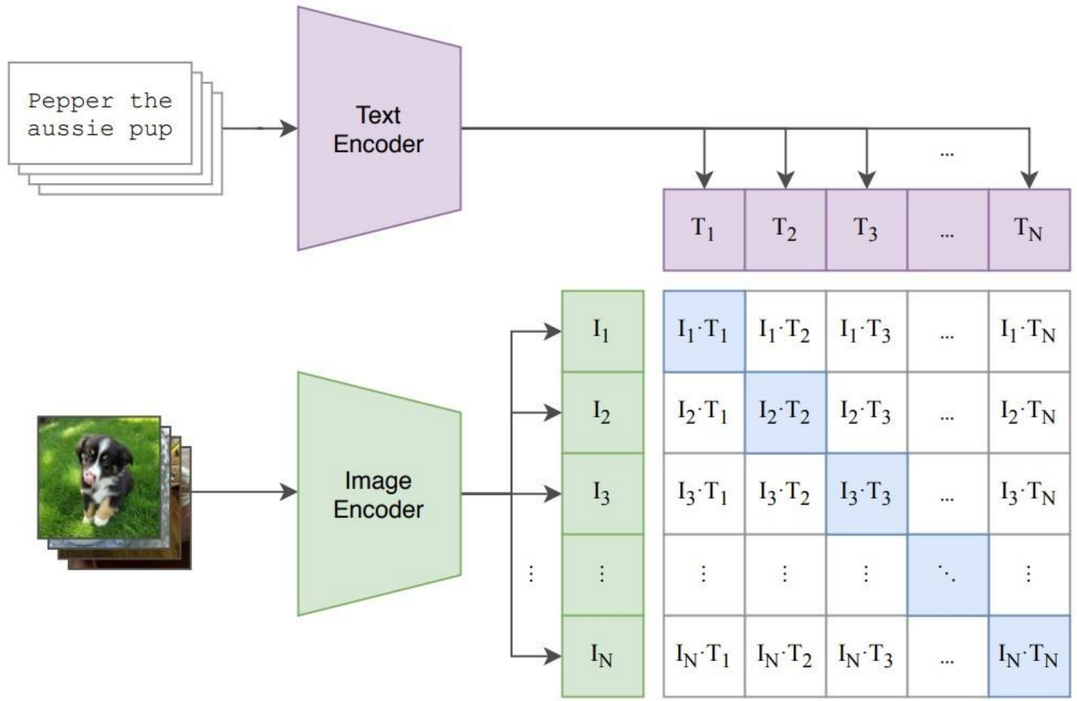
Replace the classifier in classifier guidance with a CLIP model

- Sample with a modified score:

$$\nabla_{\mathbf{x}_t}[\log p(\mathbf{x}_t|\mathbf{c}) + \omega \log p(\mathbf{c}|\mathbf{x}_t)]$$

$$= \nabla_{\mathbf{x}_t}[\log p(\mathbf{x}_t|\mathbf{c}) + \omega(\underbrace{\log p(\mathbf{c}|\mathbf{x}_t) - \log p(\mathbf{c})}_{\text{CLIP model}})]$$

$$= \nabla_{\mathbf{x}_t}[\log p(\mathbf{x}_t|\mathbf{c}) + \omega(f(\mathbf{x}_t) \cdot g(\mathbf{c}))]$$



Radford et al., “Learning Transferable Visual Models From Natural Language Supervision”, 2021.
Nichol et al., “GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models”, 2021.

GLIDE

OpenAI

- Fine-tune the model especially for inpainting: feed randomly occluded images with an additional mask channel as the input.

"an old car in a snowy forest"

"a man wearing a white hat"

Text-conditional image inpainting examples

Nichol et al., "GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models", 2021.

DALL·E 2

OpenAI

a shiba inu wearing a beret and black turtleneck

a close up of a handpalm with leaves growing from it

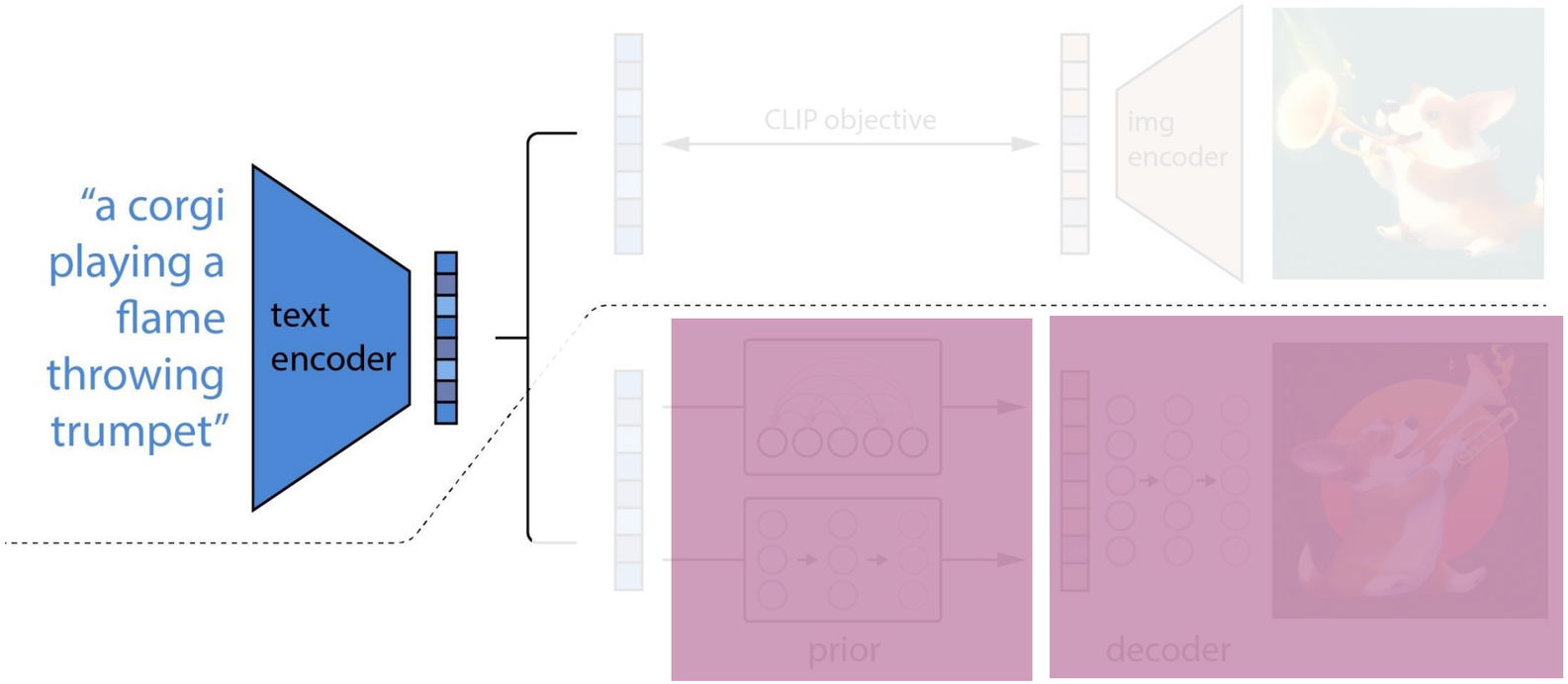
1kx1k Text-to-image generation.

Outperform DALL-E (autoregressive transformer).

Ramesh et al., “Hierarchical Text-Conditional Image Generation with CLIP Latents”, arXiv 2022.

DALL·E 2

Model components



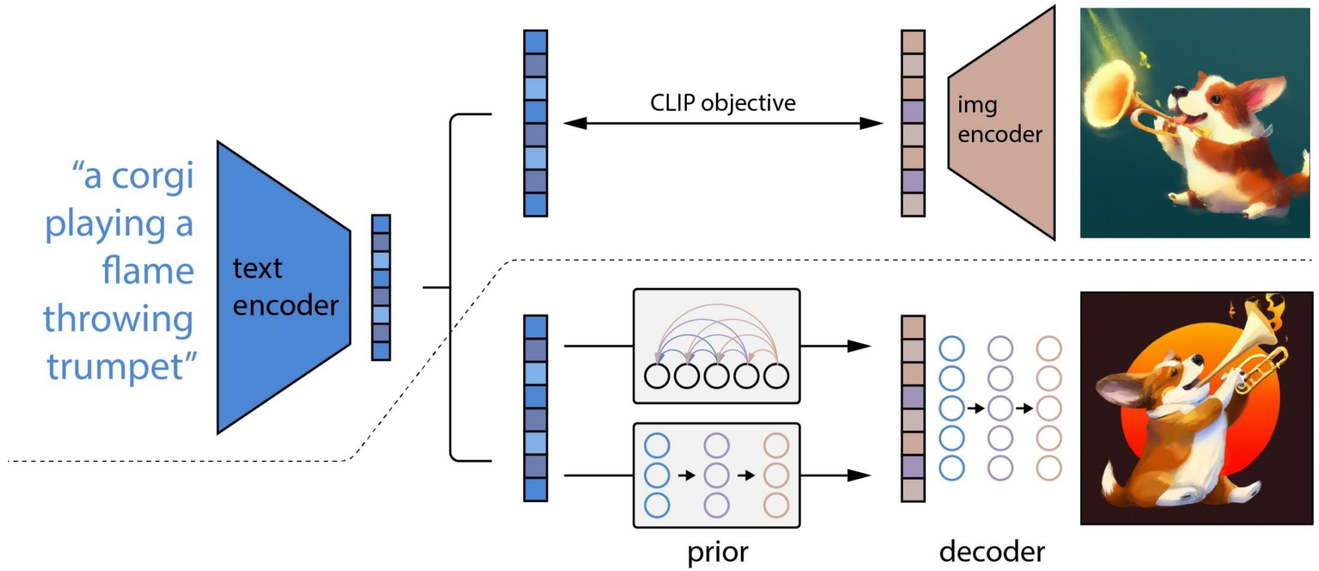
Prior: produces CLIP image embeddings conditioned on the caption.

Decoder: produces images conditioned on CLIP image embeddings and text.

Ramesh et al., "Hierarchical Text-Conditional Image Generation with CLIP Latents", arXiv 2022.

DALL·E 2

Model components



Why conditional on CLIP image embeddings?

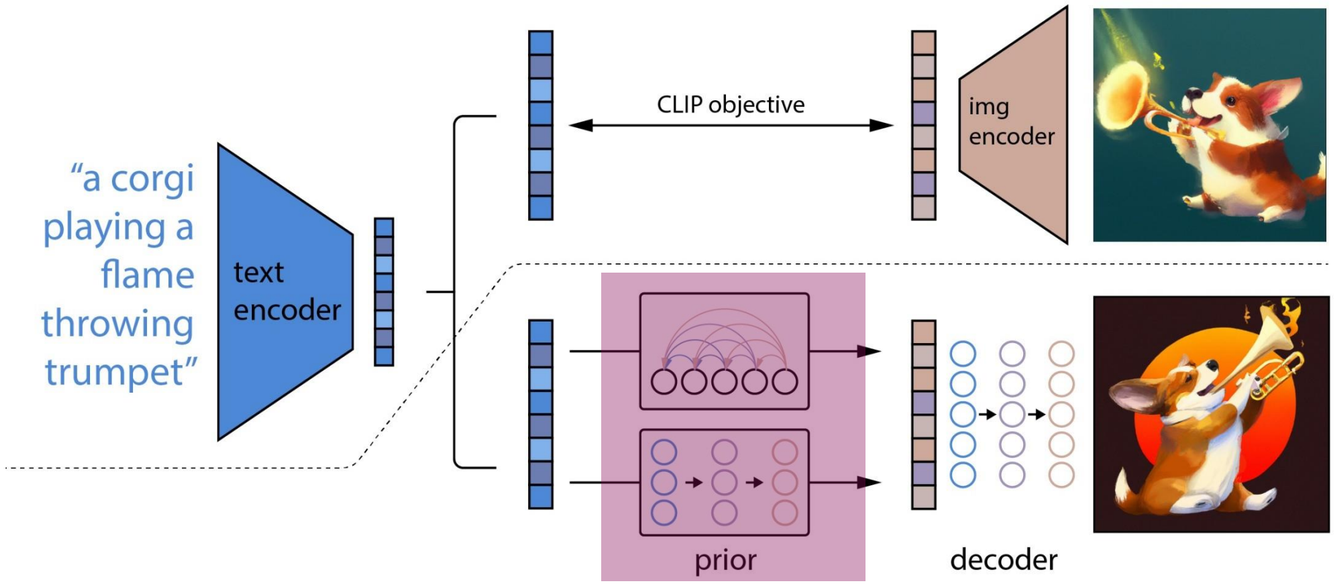
CLIP image embeddings capture high-level semantic meaning; latents in the decoder model take care of the rest.

The bipartite latent representation enables several text-guided image manipulation tasks.

Ramesh et al., "Hierarchical Text-Conditional Image Generation with CLIP Latents", arXiv 2022.

DALL·E 2

Model components (1/2): prior model



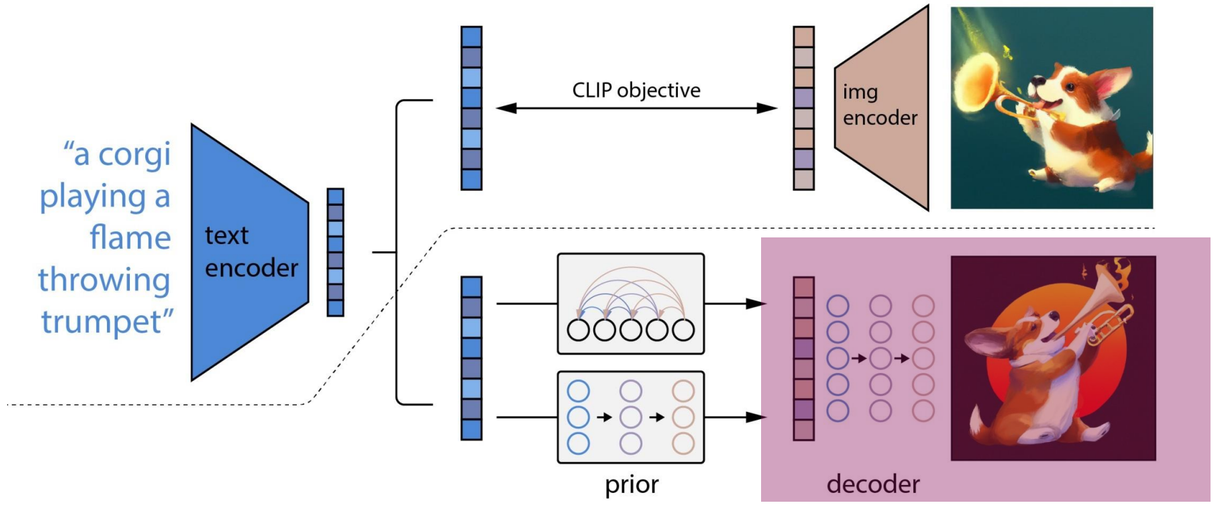
Prior: produces CLIP image embeddings conditioned on the caption.

- Option 1. autoregressive prior: quantize image embedding to a seq. of discrete codes and predict them autoregressively.
- Option 2. diffusion prior: model the continuous image embedding by diffusion models conditioned on caption.

Ramesh et al., "Hierarchical Text-Conditional Image Generation with CLIP Latents", arXiv 2022.

DALL·E 2

Model components (2/2): decoder model



Decoder: produces images conditioned on CLIP image embeddings (and text).

- Cascaded diffusion models: 1 base model (64x64), 2 super-resolution models (64x64 → 256x256, 256x256 → 1024x1024).
- Largest super-resolution model is trained on patches and takes full-res inputs at inference time.
- Classifier-free guidance and noise conditioning augmentation are important.

Ramesh et al., "Hierarchical Text-Conditional Image Generation with CLIP Latents", arXiv 2022.

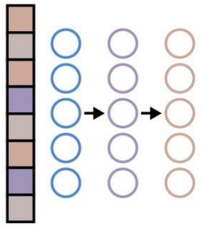
DALL·E 2

Bipartite latent representations

Bipartite latent representations $(\mathbf{z}, \mathbf{x}_T)$

$\mathbf{z}$: CLIP image embeddings

$\mathbf{x}_T$: inversion of DDIM sampler
(latents in the decoder model)



decoder

Near exact reconstruction

Ramesh et al., “Hierarchical Text-Conditional Image Generation with CLIP Latents”, arXiv 2022.

DALL·E 2

Image variations

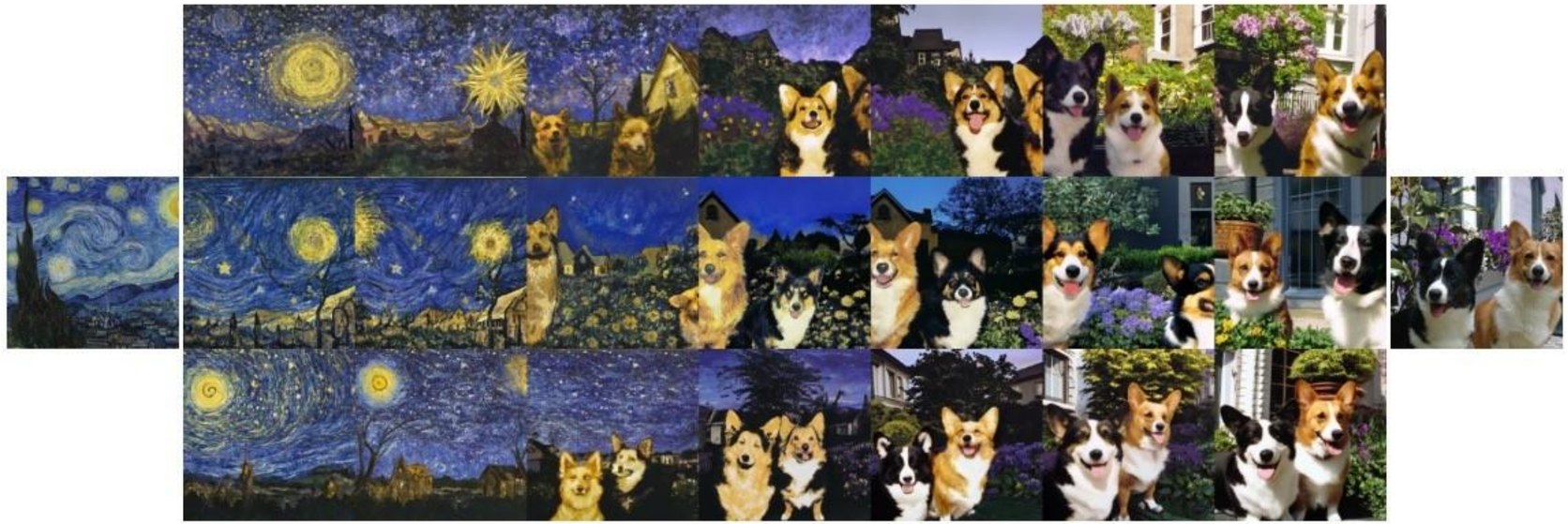
Fix the CLIP embedding $\mathbf{z}$.

Decode using different decoder latents $\mathbf{x}_T$

Ramesh et al., “Hierarchical Text-Conditional Image Generation with CLIP Latents”, arXiv 2022.

DALL·E 2

Image interpolation



Interpolate image CLIP embeddings $\mathbf{z}$.

Use different $\mathbf{x}_T$ to get different interpolation trajectories.

Ramesh et al., “Hierarchical Text-Conditional Image Generation with CLIP Latents”, arXiv 2022.

DALL·E 2

Text Diffs

a photo of a cat → an anime drawing of a super saiyan cat, artstation

a photo of a victorian house → a photo of a modern house

a photo of an adult lion → a photo of lion cub

Change the image CLIP embedding towards the difference of the text CLIP embeddings of two prompts.

Decoder latent is kept as a constant.

Ramesh et al., “Hierarchical Text-Conditional Image Generation with CLIP Latents”, arXiv 2022.

Imagen

Google Research, Brain team

Input: text; Output: 1kx1k images

- An unprecedented degree of photorealism
  - SOTA automatic scores and human ratings
- A deep level of language understanding
- Extremely simple
  - no latent space, no quantization

A brain riding a rocketship heading towards the moon.

Saharia et al., “Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding”, arXiv 2022.

Imagen

Google Research, Brain team

A photo of a Shiba Inu dog with a backpack riding a bike. It is wearing sunglasses and a beach hat.

Saharia et al., “Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding”, arXiv 2022.

Imagen

Google Research, Brain team

A dragon fruit wearing karate belt in the snow.

Saharia et al., “Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding”, arXiv 2022.

Imagen

Google Research, Brain team

A relaxed garlic with a blindfold reading a newspaper while floating in a pool of tomato soup.

Saharia et al., “Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding”, arXiv 2022.

Imagen

Google Research, Brain team

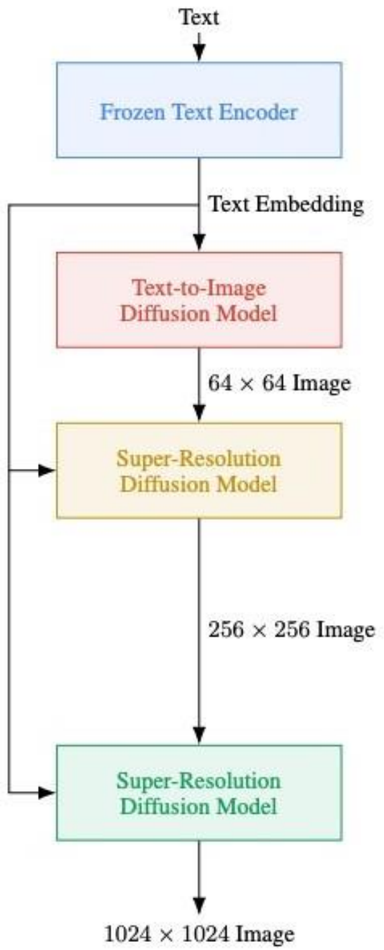
A cute hand-knitted koala wearing a sweater with 'CVPR' written on it.

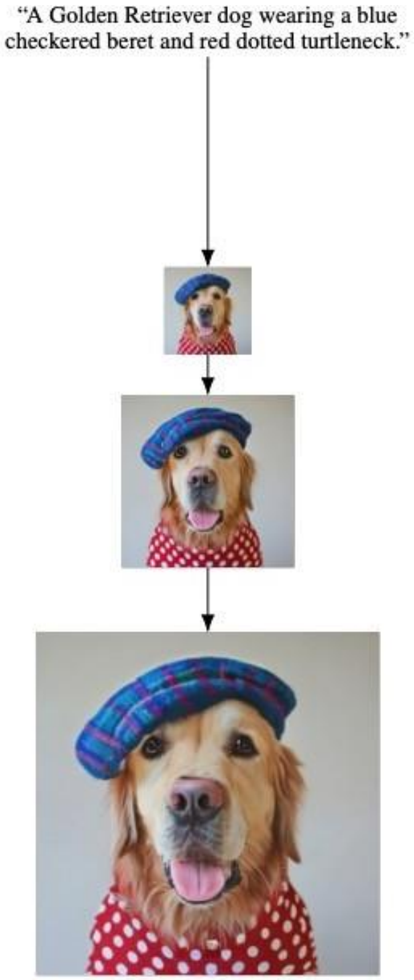
Saharia et al., “Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding”, arXiv 2022.

Imagen

Key modeling components:

- Cascaded diffusion models
- Classifier-free guidance and dynamic thresholding.
- Frozen large pretrained language models as text encoders. (T5-XXL)



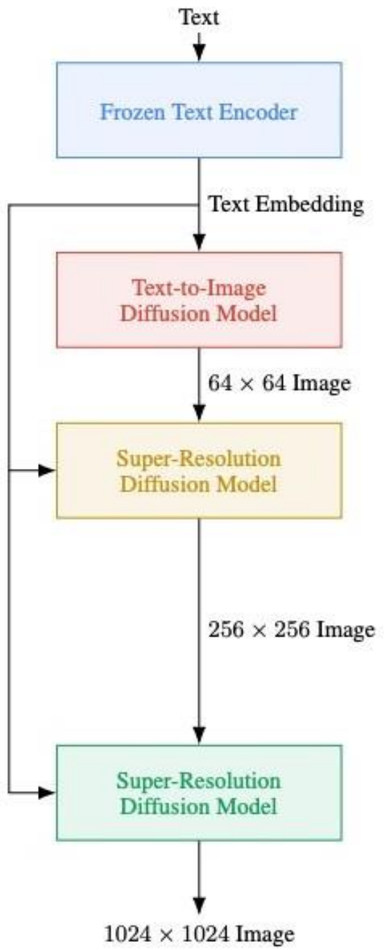


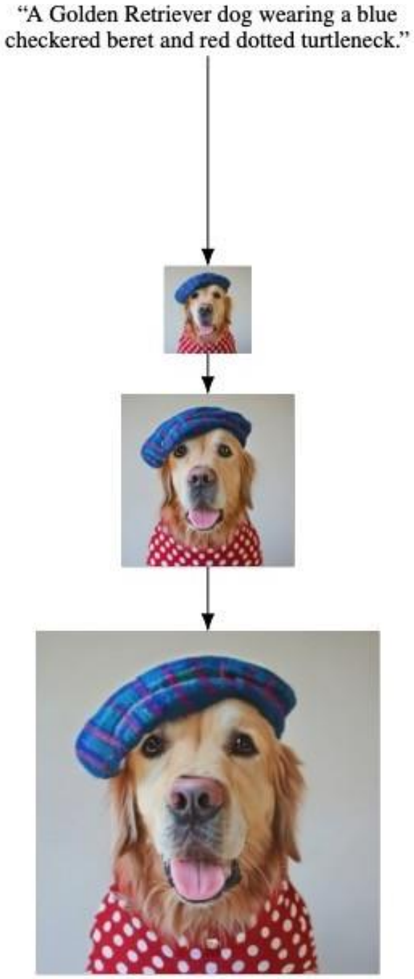
Saharia et al., "Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding", arXiv 2022.

Imagen

Key observations:

- Beneficial to use text conditioning for all super-res models.
    - Noise conditioning augmentation weakens information from low-res models, thus needs text conditioning as extra information input.
- Scaling text encoder is extremely efficient.
    - More important than scaling diffusion model size.
- Human raters prefer T5-XXL as the text encoder over CLIP encoder on DrawBench.



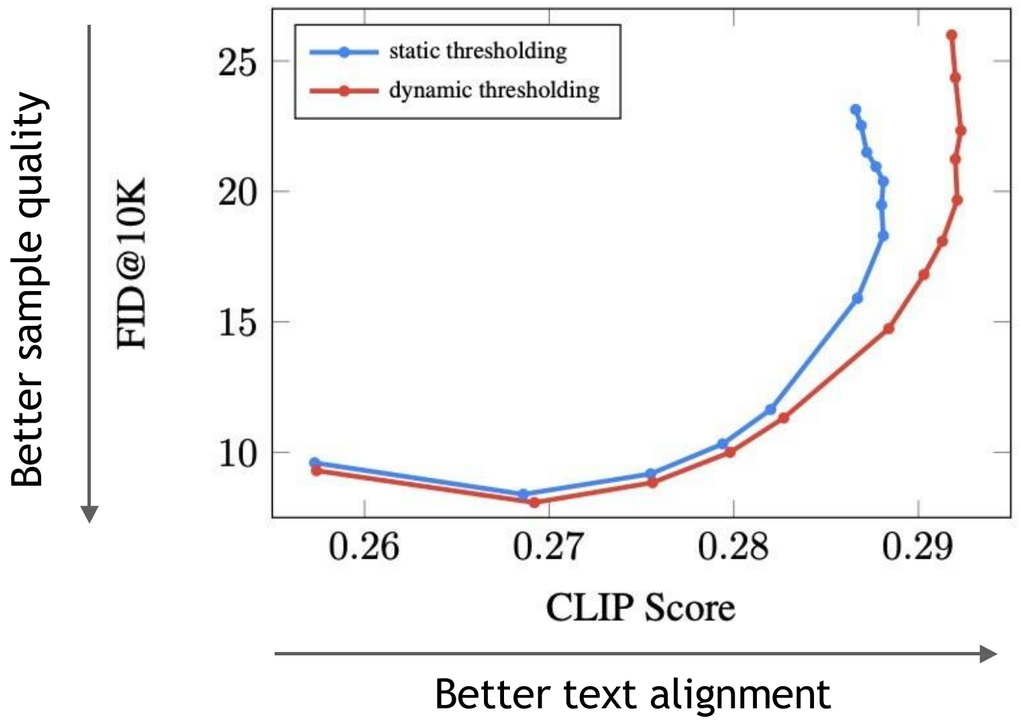


Saharia et al., "Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding", arXiv 2022.

Imagen

Dynamic thresholding

- Large classifier-free guidance weights → better text alignment, worse image quality



Saharia et al., “Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding”, arXiv 2022.

Imagen

Dynamic thresholding

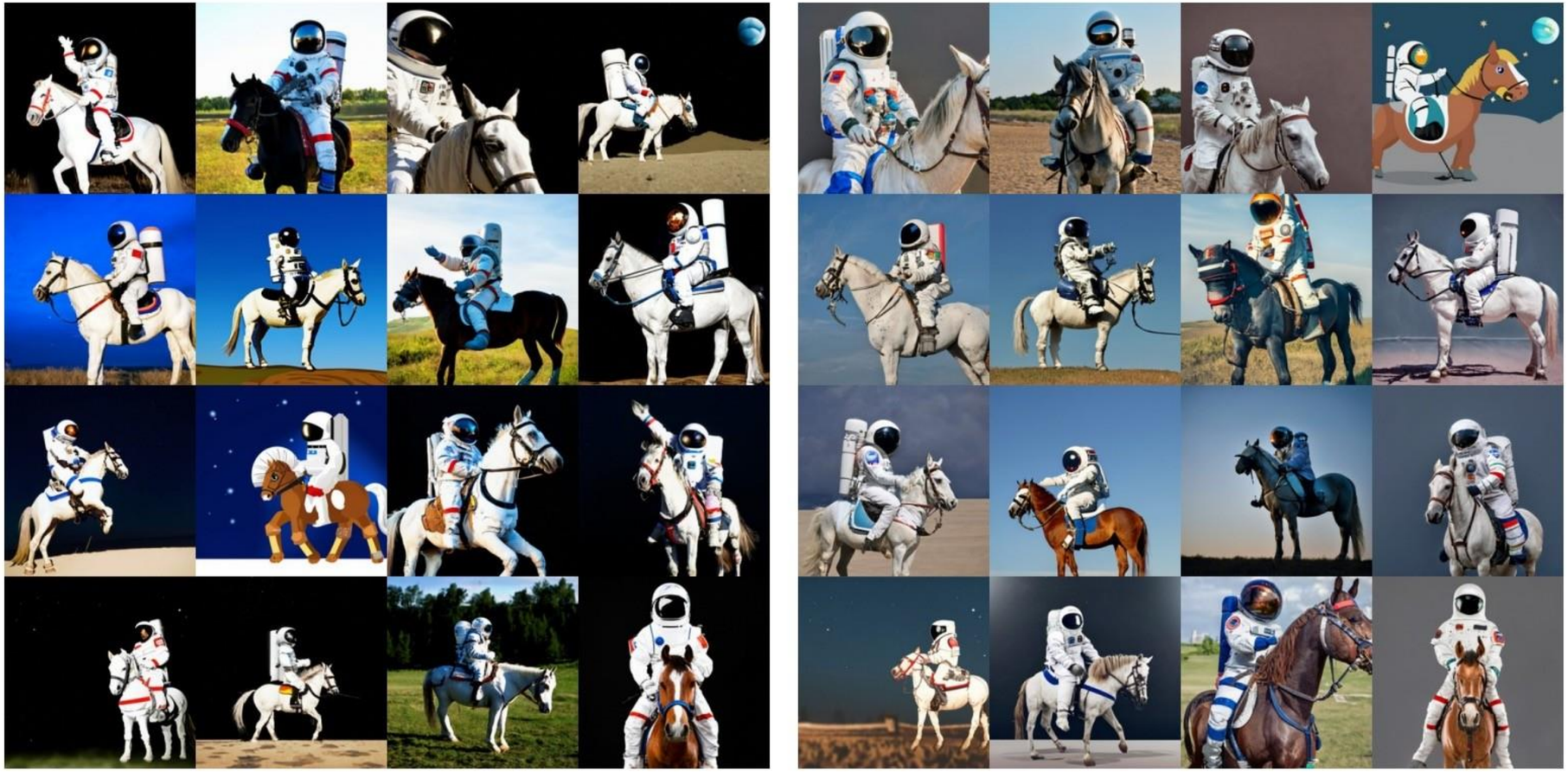
- Large classifier-free guidance weights → better text alignment, worse image quality
- Hypothesis : at large guidance weight, the generated images are saturated due to the very large gradient updates during sampling
- Solution – dynamic thresholding: adjusts the pixel values of samples at each sampling step to be within a dynamic range computed over the statistics of the current samples.

Saharia et al., “Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding”, arXiv 2022.

Imagen

Dynamic thresholding

- Large cl
- Hypothe … ry large gra
- Solution … p to be within a



Saharia et al., “Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding”, arXiv 2022.

Imagen

DrawBench: new benchmark for text-to-image evaluations

- A set of 200 prompts to evaluate text-to-image models across multiple dimensions.
  - E.g., the ability to faithfully render different colors, numbers of objects, spatial relations, text in the scene, unusual interactions between objects.
  - Contains complex prompts, e.g, long and intricate descriptions, rare words, misspelled prompts.

Saharia et al., “Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding”, arXiv 2022.

Imagen

DrawBench: new benchmark for text-to-image evaluations

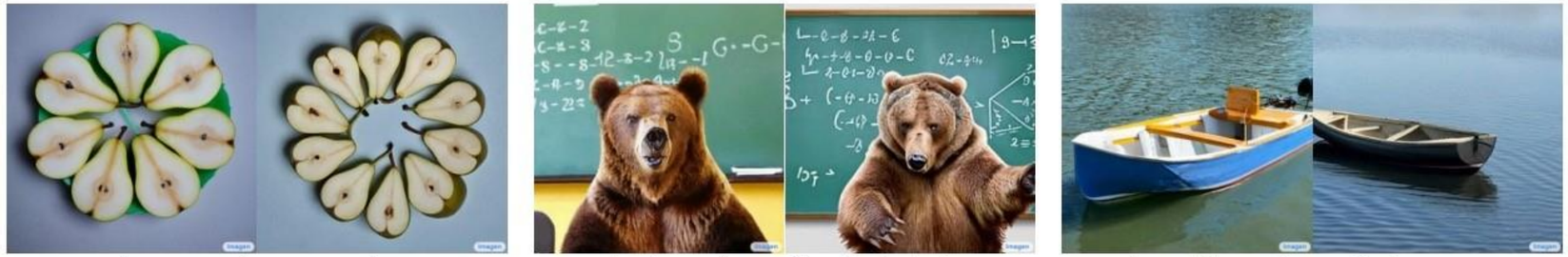
- A set of 200 prc
  - E.g., the
    the scene
  - Contains

A brown bird and a blue bear. One cat and two dogs sitting on the grass. A sign that says 'NeurIPS'.

lations, text in

lled prompts.

A small blue book sitting on a large red book. A blue coloured pizza. A wine glass on top of a dog.



A pear cut into seven pieces arranged in a ring. A photo of a confused grizzly bear in calculus class. A small vessel propelled on water by oars, sails, or an engine.

Saharia et al., "Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding", arXiv 2022.

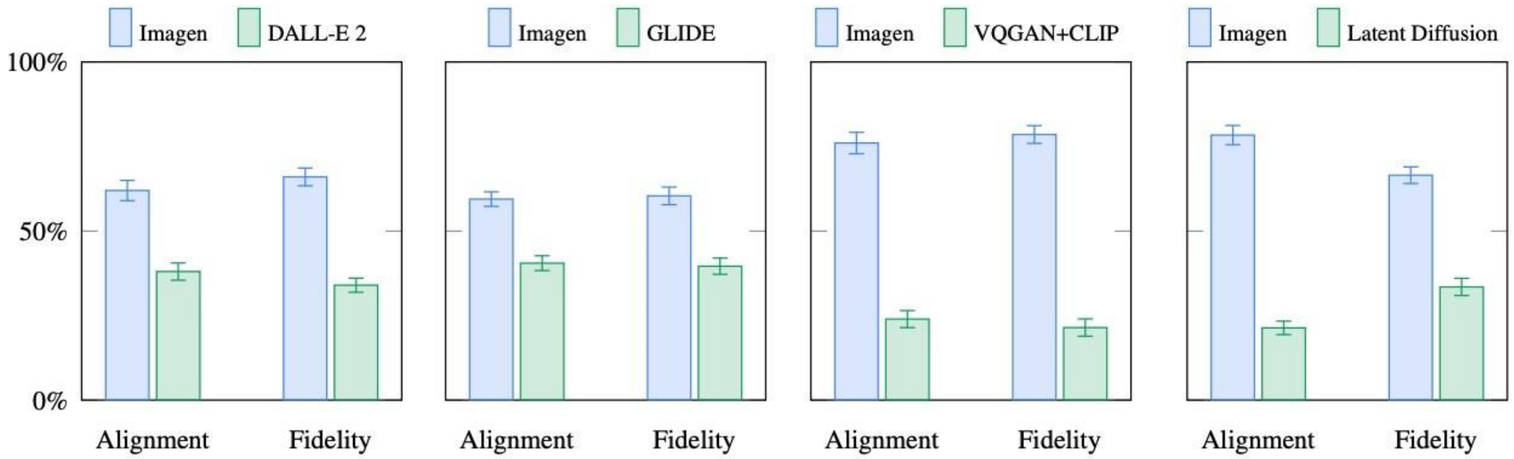
Imagen

Evaluations

Imagen got SOTA automatic evaluation scores on COCO dataset

Model	FID-30K	Zero-shot FID-30K
AttnGAN [76]	35.49	
DM-GAN [83]	32.64	
DF-GAN [69]	21.42	
DM-GAN + CL [78]	20.79	
XMC-GAN [81]	9.33	
LAFITE [82]	8.12	
Make-A-Scene [22]	7.55	
DALL-E [53]		17.89
LAFITE [82]		26.94
GLIDE [41]		12.24
DALL-E 2 [54]		10.39
Imagen (Our Work)		**7.27**

Imagen is preferred over recent work by human raters in sample quality and image-text alignment on DrawBench.



Saharia et al., “Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding”, arXiv 2022.

Thanks for listening!